Final Year Design Project

Local Business Support Platform

By

Yasir Mehmood 076984

Hafiz Bilal Khalil 077018

Project ID: BSIT22-G3

Under the supervision of

Prof. Imran Shafique

Bachelor of Science in [IT] (2022-2026)

DEPARTMENT OF INFORMATION TECHNOLOGY,

GOVT GRADUATE ISLAMIA COLLEGE, GUJRANWALA

A project presented to

University of the Punjab, Lahore

In partial fulfilment of the requirement for the degree of

Bachelor of Science in [IT] (2022-2026)

By

Yasir Mehmood 076984

Hafiz Bilal Khalil 077018

DEPARTMENT OF INFORMATION TECHNOLOGY,

GOVT GRADUATE ISLAMIA COLLEGE, GUJRANWALA

DECLARATION

We declare that this Final Year Project titled" Local Business Support Platform" is our own work and has been done by us. We have not copied any portion of this project from any source and all the matter given in this report is the outcome of our work.

We also confirm that the work is not being submitted by any other degree or qualification. If any portion of this project is found to be copied or misleading, then the responsibly lies on us.

Signature: ---------- Signature: ----------

Yasir Mehmood [076984] Hafiz Bilal Khalil [077018]

CERTIFICATE OF APPROVAL

It is clarified that the Final Year Project title **"Local Business Support Platform"** has been successfully completed by **Yasir Mehmood (076984)** and **Hafiz Bilal Khalil (077018)** under my supervision. In my opinion, the project meets the required standards in terms of quality, implementation and documentation for the degree of Bachelor of Science in **[IT].**

Signature: ---------- **FYDP**

Supervisor:

Signatures (Faculty Advisory Committee (FAC)

Signatures			
Name	**Prof Usman Ahmad**	**Prof Muhammad Asif**	**Prof Imran Shafique**
	FAC1	FAC2	FAC3

Signature: ---------- **Head**

of FYDP Coordination Office:

Signature: ---------- Dated: _____

Chairperson, Department of [IT]

Executive Summary

Local Business Support Platform is a web-based system that allows users to look for local businesses and services around them. Many small businesses face difficulty in reaching customers due to limited online presence, while users often struggle to find reliable and updated information.

The project provides a solution by creating a platform in which users can search for businesses, view their details, read reviews and locate them on an interactive map, built using OpenStreetMap.

Acknowledgement

We are thankful to Allah Almighty for giving us the strength and ability to complete this project successfully.

We would like to thank our supervisor **Prof. Imran Shafique,** for his valuable guidance and encouragement throughout the development of the project.

We are also thankful to our teachers and friends who helped us during the project. Finally, we would like to thank our family for their continuous support and motivation.

Signature: ----------

Signature: ----------

Yasir Mehmood [076984]

Hafiz Bilal Khalil [077018]

Abbreviations

All the abbreviations and acronyms used in your FYP along with their respective brief description.

Abbreviation	Full Form	Description
API	Application Programming Interface	Used to connect our system with external services such as the OpenStreetMap mapping service.
UI	User Interface	Refers to the design and screens that users interact with (search page, dashboard etc.)
DB	Database	Used to store all system data such as users, businesses, reviews and reports.
IT	Information Technology	Represents the field under which this project is developed.
HTML	Hypertext Markup Language	Used to create the basic structure of web pages in the front end.
CSS	Cascading Style Sheets	Used to design and style the user interface to make it attractive and user friendly.
JS	JavaScript	Used to add functionality and interactivity in the system (like search, button etc.)
GPS	Global Positioning System	Used for location tracking to help user's fine nearby businesses.
SQL	Structured Query Language	Used to manage and query the database (MySQL) for storing and retrieving data.
CRUD	Create, Read, Update, Delete	Represents basic operations performed on business listings and user data.
JWT	JSON Web Token	Used for secure login and authentication of users in the system.

Table Of Contents

1. Introduction 1
   1.1 Problem Statement 1
   1.2 Problem Solution 1
   1.3 Objectives of the Proposed System 2
   1.4 Scope 2
   1.5 System Component 3
      1.5.1 Module 1: User Module 3
      1.5.2 Module 2: Business Owner Module 3
      1.5.3 Module 3: Admin Module 4
   1.6 Related System Analysis/Literature Review 4
   1.7 Vision Statement 4
   1.8 System Limitations and Constraints 5
   1.9 Tools and Technologies 6
   1.10 project deliverables 6
   1.11 Project planning 8
   1.12 Summary 8
2. Analysis 10
   2.1 User classes and characteristics 10
   2.2 Requirement Identifying Technique 10
   2.3 Functional Requirements 11
      2.3.1 Functional requirement X 13
   2.4 Non-Functional Requirements 18
   2.5 Use Case Analysis 20
      2.5.1 Use Case 1 20
      2.5.2 Use Case 2 21
      2.5.3 Use Case 3 21
      2.5.4 Use Case 4 22
      2.5.5 Use Case 5 22
      2.5.6 Use Case 6 23
      2.5.7 Use Case 7 24
      2.5.8 Use Case 8 24
      2.5.9 Use Case 9 25
      2.5.10 Use Case 10 25
      2.5.11 Use Case 11 26
      2.5.12 Use Case 12 27

2.5.13 Use Case Diagram....28
2.6 External Interface Requirements....29
2.6.1 User Interface Requirements....29
2.6.2 Software Interface....29
2.6.3 Hardware Interface....29
2.7 Summary....30
3. System Design....31
3.1 Design Considerations....32
3.2 Design Models....33
3.3 Architectural Design....45
3.4 Data Design....49
3.4.1 Data Dictionary....53
3.5 User Interface Design....58
3.5.1 Screen Images....59
3.5.2 Screen Objects and Actions....63
3.6 Design Decisions....64
3.7 Summary....65
4. Implementation....67
4.1 Algorithm....67
4.2 External APIs / SDKs....69
4.3 Code Repository....70
4.3.1 Metrices of the Git Repository:....70
4.4 Summary....71
Chapter 5: Testing and Evaluation....73
5.1 Unit Testing (UT)....73
5.2 Functional Testing....77
5.3 Integration Testing....81
5.4 Performance Testing....83
5.5 Summary....85
6. Introduction....86
6.1 Conversion Method....86
6.2 Deployment....87
6.2.1 Data Conversion....88
6.2.2 Training....89
6.3 Post Deployment Testing....93
6.4 Challenges....95

6.5 Summary 97
7. Conclusion 98
7.1 Evaluation 98
7.2 Requirements Traceability Matrix 99
7.3 Conclusion 100
7.4 Future Work 101
7.4.1 Payment Integration 101
7.4.2 Delivery Management System 101
7.4.3 Mobile Application 101
7.4.4 Chat System Between Users and Business Owners 102
7.4.5 Multi-City Expansion 102
References 102
Appendix A: Use Case Description 104
Coding Standard 111
Appendix-B General Coding Standard & Guidelines 111
Appendix C: Application Prototype 114

List Of Figures

Figure 2-1: Use Case Diagram 28
Figure 3-1: Class Diagram 34
Figure 3-2: Sequence Diagram 1 35
Figure 3-3: Sequence Diagram 2 36
Figure 3-4: Sequence Diagram 3 37
Figure 3-5: Sequence Diagram 4 38
Figure 3-6: View Pending Businesses 39
Figure 3-7: Verify Business Process 40
Figure 3-8: Sequence Diagram 5 40
Figure 3-9: Sequence Diagram 6 41
Figure 3-10: Sequence Diagram 7 42
Figure 3-11: Sequence Diagram 8 43
Figure 3-13: State Transition Diagram 2 45
Figure 3-14: UML Class Relationship Diagram 46
Figure 3-15: UML Component Diagram 47
Figure 3-16: Model View Control Diagram 48
Figure C-1: Landing Page (Homepage) 115
Figure C-2: Login Screen 116
Figure C-3: Registration Screen 117
Figure C-4:Search Screen 118
Figure C-5: Business Detail Screen 119
Figure C-6: Admin Dashboard 120
Figure C-7: Admin Dashboard - Overview and Pending Approvals 121

List Of Table

Table 2-10: Description of FR-8: Add Promotion 16
Table 2-11: Description of FR-9: Report Business 17
Table 2-12: Description of FR-10: Admin Approval 17
Table 2-13: Description of FR-11: Manage Reports 18
Table 2-14: Description of FR-12: View Map Location 18
Table 2-15: Non-Functional Requirement Table 19
Table 2-16: User Registration..... 20
Table 2-17: User Login 21
Table 2-18: Search Business 22
Table 2-19: View Business Details 22
Table 2-20: Add Business Listing 23
Table 2-21: Edit Business Listing 23
Table 2-22: Delete Business Listing 24
Table 2-23: Submit Review 25
Table 2-24: View Reviews..... 25
Table 2-25: Admin Approval 26
Table 2-26: Notification System..... 26
Table 2-27: Location Services 27
Table 3-1: Layered (Three-tier) Pattern Mapping 49
Table 3-2: MVC (Model-View-Controller) Pattern Mapping (on the backend) 50
Table 3-3: Alphabetical Two-Column Format (Terminology & Descriptions) 55
Table 4-1: Search Local Business Algorithm 69
Table 4-2: Add Business Listing Algorithm 70
Table 4-3: Submit Review Algorithm 70
Table 4-4: External APIs and SDKs 71
Table 5-1: UT-01: User Registration 75
Table 5-2: UT-02: User Login (Valid) 76
Table 5-3: UT-03: User Login (Invalid Password) 76
Table 5-4: UT-04: Add Business Validation 77
Table 5-5: UT-05: Geocode Address 78
Table 5-6: UT-06: Search by Category 78
Table 5-7: FT-01: Search with Location 79
Table 5-8: FT-02: Business Owner Adds Listing 80
Table 5-9: FT-03: Customer Writes Review 80
Table 5-10: FT-04: Admin Approves Pending Business 81
Table 5-11: FT-05: Report on ❼Fake Business 81
Table 5-12: FT-06: Update Profile 82
Table 5-13: IT-01: Add ❼Approve ❼Search Flow 83
Table 5-14: IT-02: Review Updates Business Rating 83
Table 5-15: IT-03: Report Triggers Admin Notification 84
Table 5-16: PT-01: Search API Load Test 85
Table 5-17: PT-02: Home Page Load Speed 85
Table 5-18: PT-03: Add Business API Response Time 86
Table 6-1: Deployment Steps 88
Table 6-2: Data Conversion Steps 89

Table 6-3: User Manual for Business Owners – Adding a Listing 91
Table 6-4: User Manual for Normal Users – Searching and Writing a Review 93
Table 6-5: Post-Deployment Test Cases 95
Table 6-6: Deployment Challenges and Solutions 97
Table 7-1: Evaluation of Project Objectives 99
Table 7-2: Requirements Traceability Matrix 100
Table 7-3: Correlation of Objectives and Functionality 101
Table A-1: Use Case UC-01 – Register Account 105
Table A-2: Use Case UC-02 – Add New Business Listing 106
Table A-3: Use Case UC-03 – Search Businesses 107
Table A-4: Use Case UC-04 – Write a Review 108
Table A-5: Use Case UC-06 – Approve or Reject Business Listing 109

Chapter 1: Introduction

1. Introduction

This chapter provides a complete overview of our final year project titled **"Local Business Support Platform".** The main purpose of this chapter is to explain the problem, proposed solutions, objectives, scope and overall direction of the system. This chapter helps in understanding why this system is needed and also the benefits to the users, the business owners and the local community.

1.1 Problem Statement

In today's digital area, many local businesses are still struggling to reach customers because they are not online and limited sources. Most small businesses do not have proper platforms where they can promote their services or products effectively. On the other hand, customers also face difficulty in finding reliable nearby businesses, especially when accurate and updated information is not available.

Existing systems like online directories or map services provide basic information, but they often lack updated data, user reviews, and proper interaction between users and business owners. Due to this, customers may not trust the information, and businesses lose potential customers.

The system will use technologies like React.js for frontend, Node.js for backend, MySQL for databases and OpenStreetMap (via the Leaflet library) for location-based features. Overall, the system will provide a simple and efficient solution to support local businesses and help users find reliable services easily.

1.2 Problem Solution

To solve the above problems, we propose a web-based system called **Local Business Support Platform**. This system will act as a bridge between users, business owners, and admins.

The system will allow users to search for nearby businesses, view their details, read reviews, and see their location on a map. Business owners will be able to register their businesses, add details, manage listings, and promote their services. Admin will monitor the system, verify business listings, and ensure that the information remains authentic.

The system will use technologies like React.js for frontend, Node.js for backend, MySQL for databases and OpenStreetMap (via the Leaflet library) for location-based features. Overall, the system will provide a simple and efficient solution to support local businesses and help users find reliable services easily.

1.3 Objectives of the Proposed System

1. Create a web-based platform that allows users to discover local businesses and view results sorted by distance from their current location using a Near Me feature.
2. Allow users to search for businesses using filters such as category, business name and location with search results appearing within 3 seconds.
3. Allow users to read reviews and write their own reviews with a rating from 1 to 5 stars for any business they have visited.
4. Allow business owners to register, create their business listing, and manage their profile including editing details and adding promotions without any technical help.
5. Implement an admin panel where the admin can verify new business listings within 24 hours of submission and manage any user reports.
6. Integrate an interactive mapping solution using OpenStreetMap and the Leaflet library so that users can see the exact location of each business on a map and get directions from their current location.
7. Expand the platform beyond a single city so that business owners and customers from multiple major cities across Pakistan can use the same system.
8. Implement a live photo upload feature during business registration, encouraging photos to be captured directly from the device camera rather than selected from a gallery, to improve the authenticity of listings.
9. Provide a reporting system where users can report suspicious businesses or fake reviews, and admin can act within 48 hours.
10. Ensure the platform is fully responsive so that users can access it from mobile phones, tablets and desktop computers without any layout issues.

1.4 Scope

The scope of this project includes developing a web-based platform that connects users, business owners and admins in one system. The system will include features like business listing, searching, filtering, reviews, reporting and map-based location viewing.

The system covers multiple cities across Pakistan rather than being limited to a single city. It will not include online payment or delivery services. It will mainly focus on helping users find businesses and helping businesses promote themselves.

The major stakeholders of the system include users, business owners and admin. The project will be developed within limited time and resources and priority will be given to core features like search, listing, and verification.

1.5 System Component

The proposed system is divided into three main modules or components. Each module has a specific set of features and responsibilities.

1.5.1 Module 1: User Module

- User registration and login with email and password
- Search businesses by name, category or location
- Filter search results by category and city, and sort results by distance using the Near Me feature
- View business details including name, address, phone, description and opening hours
- See business location on an interactive map built with OpenStreetMap
- Read reviews and ratings written by other users
- Write a new review with a rating from 1 to 5 stars
- Report a business if information seems fake or suspicious

1.5.2 Module 2: Business Owner Module

- Business owner registration and login
- Add a new business listing with name, address, category, description, opening hours and live photo
- Edit existing business listing details anytime
- Delete a business listing (the listing, along with its reviews and promotions is permanently removed)
- Add promotions or special offers that appear on the business profile
- View all reviews written by users about their business

- Receive email notifications when listing is approved or rejected

1.5.3 Module 3: Admin Module

- Admin login with secure authentication
- View all pending business submissions waiting for approval
- Approve or reject business listings with a reason for rejection
- View all active businesses on the platform
- View all user reports submitted against businesses
- Investigate reported businesses and mark report as resolved
- Suspend a business listing if it violates platform rules
- Delete a business listing permanently
- View all registered users and business owners

1.6 Related System Analysis/Literature Review

Some existing systems provide similar services but have certain limitations.

Table 1-1: Related System Analysis with Proposed Project Solution

Website Name	**Weakness**	**Solution**
Google Maps	Limited interaction and reviews system.	Provide detailed review and rating system.
Lookup.pk	Outdated information.	Allow owners to update their data.
brown book	Paid features for interaction.	Provide free and simple access.

1.7 Vision Statement

A vision statement for Local businesses Support Platform:

For users who want to easily find nearby local businesses and services

Who faces difficulty in accessing accurate and updated information about local shops and service providers

The Local Business Support Platform

Is a web-based application

That allows users to search for businesses, view details, read reviews, and locate them on a map while also enabling business owners to promote and manage their services,

Unlike traditional business directories and existing systems that provide limited or outdated information and lack user interaction,

Our product provides an interactive, user-friendly, and reliable platform with updated data, review system, and admin verification to ensure authenticity and better user experience.

1.8 System Limitations and Constraints

Limitations:

- Data accuracy depends on business owner
- Map pin accuracy depends on the free OpenStreetMap geocoding service, which can occasionally fail for uncommon or incomplete addresses
- Possible fake reviews or spam

Constraints:

- Limited time for development
- Limited resources and technology
- Dependency on internet connectivity and third-party mapping services

1.9 Tools and Technologies

Table 1-2: Tools and Technologies for Proposed Project

Tools Libraries And Technology	**Tools**	**Version**	**Rationale**
	Visual Studio Code	1.89	Code editing and debugging
	Libraries	**Version**	**Rationale**
	React.js	18.2.0	Front-end framework for building a fast, component-based UI
	Express.js	.18.2	Back-end framework for Node.js
	Leaflet + OpenStreetMap	1.9	Open-source mapping library used for map display and geo location, in place of a paid mapping API
	Nodemailer	6.x	Sends approval, rejection and password-reset emails through Gmail SMTP
	Multer	1.4	Handles live photo upload from the business owner dashboard
	JWT (jsonwebtoken) bcryptjs	9.x / 2.x	Token-based authentication and password hashing
	React Router DOM	6.x	Client-side routing so each page has its own URL
	Technology	**Version**	**Rationale**
	MySQL	8.0	Database for Storing Data
	HTML, CSS, JS	5,3, ES 2025	For front end design

1.10 project deliverables

Here is the list of project deliverables for local business support.

1. Planning document Purpose:

This document explains the overall planning of the project, including project goals, timeline and how

The work will be managed step by step.

Contents:

Project scope, objectives, development phases, schedule (timeline), task distribution and basic risk planning.

Appendix in FYP report:

This document will be included as part of the initial planning section to show how the project was organized and executed.

1. Software Requirements Specification (SRS)

Purpose:

This document defines all the functional and non-functional requirements of the system clearly.

Contents:

Use cases, user stories, system features, functional requirements, non-functional requirements, and system interfaces.

Appendix in FYP Report:

This document will provide complete details of system requirements and will be used as a reference during development and testing.

2. Design and System Specification

Purpose:

This document describes how the system is designed and how different components of the system will work together.

Contents:

System architecture, UML diagrams (use case, class, sequence), database design, ER diagrams and user interface design.

Appendix in FYP Report:

This document will help in understanding the system structure and will guide the implementation phase.

3.Prototypes

Purpose:

This document provides visual representation of the system interface and user experience.

Contents:

Screenshots or designs of web pages such as login pages, dashboard, business listing page and search interface.

Appendix in FYP Report:

These prototypes will help in understanding the look and feel of the system before development.

4. Final Report Purpose:

This document provides complete documentation of the project from start to end.

Contents:

All chapters include introduction, analysis, design, implementation, testing and conclusion.

Appendix in FYP Report:

This will be the main document submitted for evaluation.

1.11 Project planning

Gantt chart representing the plans of carrying out the project activities, representing the milestones as well as resources required for each task:

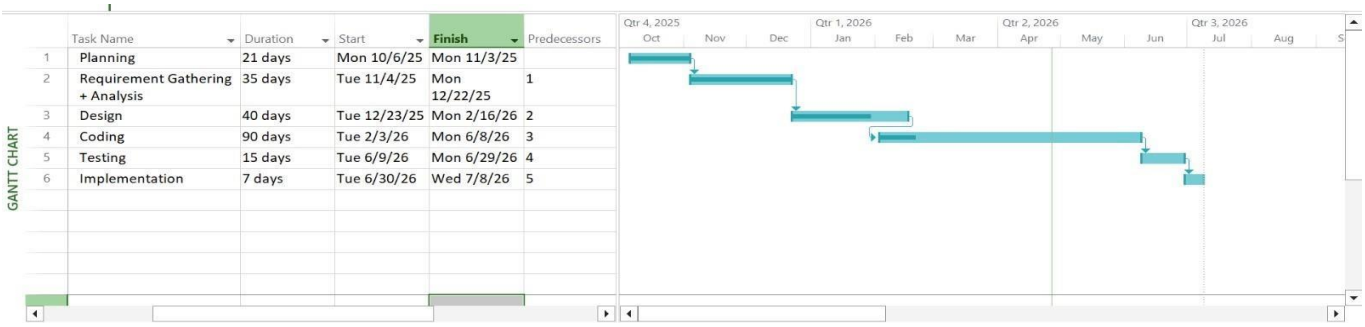


Figure 1-1: Project Gantt Chart

1.12 Summary

Here is summary of key points for local business support platform:

Key points

1. Objectives

To develop a web platform that helps users find nearby businesses and allows business owners to promote their services easily.

2. Features

Includes search and filter options, map-based location, review system, business listings and admin verification.

3. Benefits

Users get quick and reliable information, while businesses gain more visibility and customers.

4. Technologies

Built using React.js (frontend), Node.js/Express (backend), MySQL (database) and OpenStreetMap for location.

Chapter 2: Requirement Analysis

2. Analysis

This chapter explains the system requirements of the Local Business Support Platform. It describes different users of the system, how requirements were collected and what functionalities the system must provide. It also explains system quality requirements and external interfaces.

2.1 User classes and characteristics

These are the user classes and their characteristics are as follows:

Table 2-1: User Classes and Characteristics for Proposed Project

User Class	**characteristics**
Normal user	Basic user of system, search business, read /write reviews, user map
Business Owner	Registers business, manages listings, adds promotions, views reviews
Admin	Verifies businesses, manage reports, control system security
Guest user	I can search and view businesses but cannot post reviews

2.2 Requirement Identifying Technique

We used Use Case Technique to identify system requirements.

Table 2-2: User Classes and Characteristics for Proposed Project

Use Case Name	**Associated functional requirement**
User Registration	User creates a new account by entering personal details
User Login	User logs into the system using Email and password
Search Business	User searches businesses using location

View business details	User views business information, location and reviews
Write review	User submits rating and review for a business
Add business listing	Business owners add a new business to the platform
Update business listing	Business owner edits existing business details
Add promotion	Business owner adds promotion or offers
View reviews (owner)	Business owner checks reviews of their business
Report business	User report fake or inappropriate listings
Admin login	Admin log into system
Approve business	Admin approves or rejects business listings
Manage report	Admin review

2.3 Functional Requirements

This section describes the main functions of the local Business Support Platform based on the needs of users, business owners, and admin. It explains how the system helps users find local businesses, allows owners to manage listings and enables admin to control and verify the platform.

1. **User Registration**

   - The system allows users to create an account by entering name, email and password.

2. **User Login**

   - The system allows registered users to login using valid email and password.

3. **Search Business**

   - The system shall allow users to search for local businesses using keywords, category and location.
   - The system shall display a list of businesses based on user search.

4. **View Business Details**

   - The system shall show complete business details including name, category, address, location and reviews.

5. **View Map Location**

   - The system shall display business locations on an interactive map using OpenStreetMap.

6. **Write Review**

   - The system shall allow users to write reviews and give ratings to businesses.

7. **Business Report**

   - The system shall allow users to report fake or inappropriate business listings.

8. **Add Business Listing**

   - The system shall allow business owners to add new business listings.

9. **Update Business Listing**

   - The system allows business owners to edit/update their business information.

10. **Add Promotion**

    - The system shall allow business owners to add promotions or special offers.

11. **View Reviews (Owner)**

    - The system shall allow business owners to review their business.

12. **Admin Login**

    - The system allows admin to log in securely.

13. **Approving Business**

    - The system shall allow admin to approve or reject business listings.
    - The system should ensure that only approved businesses are visible to users.

14. **Manage Reports**

    - The system shall allow admin to review and manage reported businesses.

2.3.1 Functional requirement X

Here's a functional requirements specification for a Local Business Support Platform. **User Registration and Login:** User Registration is an important feature of the system that allows new users to create their account on the Local Business Support Platform. Through this feature, users can enter their basic information and get access to system services.

1. **Account Creation:** The user can create a new account by entering required details.
2. **Data Storage:** The system stores user information securely in the database.
3. **Unique Email:** Each user must register with a unique email address.
4. **Access to System:** After registration, the user can log in and use system features

2.3.2 User Registration

Table 2-3: Description of FR-1: User Registration

identifier	FR-1
Title	User Registration
Requirement	The user shall be able to create an account by entering name, email, and password
Source	user
Rationale	To allow new users to access the system
Business rule	Email must be unique
Dependencies	None
Priority	High

2.3.3 User Login

Table 2-4: Description of FR-2: User Login

Identifier	FR-2
Title	User login
Requirement	The system shall allow users to login using valid email and password

Source	User
Rationale	To authenticate users securely
Business rule	Only registered users can login
Dependencies	FR-1
Priority	High

2.3.4 Search Business

Table 2-5: Description of FR-3: Search Business

Identifier	FR-3
Title	Search business
Requirement	The user shall be able to search for businesses using keywords, Category and location
Source	User
Rationale	To help users find nearby businesses easily
Business rule	Search results should match keywords
Dependencies	None
Priority	High

2.3.5 View Business Details

Table 2-6: Description of FR-4: View Business Details

Identifier	FR-4
Title	View Business Details
Requirement	The system shall display business information including name, category, location, and reviews

Source	User
Rationale	To provide complete information about businesses
Business rule	Only approved businesses should be visible
Dependencies	FR-3
Priority	High

2.3.6 Write Review

Table 2-7: Description of FR-5: Write Review

Identifier	FR-5
Title	Write Review
Requirement	The user shall be able to submit ratings and reviews for a business
Source	User
Rationale	To allow feedback and improve trust
Business rule	One user can submit multiple reviews for different businesses
Dependencies	FR-2
Priority	High

2.3.7 Add Business Listings

Table 2-8: Description of FR-6: Add Business Listing

Identifier	FR-6
Title	Add Business Listing
Requirement	Business owners should be able to add a new business listing with details
Source	Business Owner

Rationale	To register businesses on platform
Business rule	Listing must be approved by admin before visible
Dependencies	FR-2
Priority	High

2.3.8 Update Business Listings

Table 2-9: Description of FR-7: Update Business Listing

Identifier	FR-7
Title	Update Business Listing
Requirement	Business owners shall be able to edit business details
Source	Business Owner
Rationale	To keep business information updated
Business rule	Only owners can update their listing
Dependencies	FR-6
Priority	High

2.3.9 Add Promotion

Table 2-10: Description of FR-8: Add Promotion

Identifier	FR-8
Title	Add Promotion
Requirement	Business owners shall be able to add promotional offers
Source	Business Owner
Rationale	To attract more customers
Business rule	Promotion must be linked with a business
Dependencies	FR-6
Priority	Medium

2.3.10 Business Report

Table 2-11: Description of FR-9: Report Business

Identifier	FR-9
Title	Report Business
Requirement	User shall be able to report fake or inappropriate business listings
Source	User
Rationale	To maintain system quality
Business rule	Report must include valid reason
Dependencies	FR-2
Priority	Medium

2.3.11 Admin Approval

Table 2-12: Description of FR-10: Admin Approval

Identifier	FR-10
Title	Approve Business
Requirement	Admin shall approve or reject business listings
Source	Admin
Rationale	To ensure authenticity of businesses
Business rule	Only admin can approve listings
Dependencies	FR-6
Priority	High

2.3.12 Manage Reports

Table 2-13: Description of FR-11: Manage Reports

Identifier	FR-11
Title	Manage Reports
Requirement	Admin shall review and take action on reported businesses

Source	Admin
Rationale	To control fake or harmful content
Business rule	Admin decision is final
Dependencies	FR-9
Priority	High

2.3.13 View Map Location

Table 2-14: Description of FR-12: View Map Location

Identifier	FR-12
Title	View Map
Requirement	The system shall display business locations on map using coordinates
Source	User
Rationale	To help users find exact location
Business rule	Requires the OpenStreetMap geocoding service to be reachable
Dependencies	FR-3
Priority	High

2.4 Non-Functional Requirements

These quality requirements should be specific, quantitative, and verifiable.

1.Performance Requirements

The system should respond quickly when users search for businesses or load pages.

2. Availability Requirements

The system should be available most of the time so users can access it anytime.

3. Reliability Requirements

The system should work correctly without errors and provide accurate results.

4. Security Requirements

The system should protect user data and allow only authorized access.

5. Maintainability Requirements

The system should be easy to update, fix, and improve in the future.

6. Usability Requirements

The system should be simple and easy to use for all types of users.

Table 2-15: Non-Functional Requirement Table

Identifier	Category	Requirement description	Module	priority
NFR-01	Performance	The system should load each page within 2– 3 seconds for normal use (up to 100 users at the same time).	Search & Dashboard Module	High
NFR-02	Usability	A new user should be able to use basic features (login, search, submit data) within 5 minutes without any training.	User Interface Module	High
NFR-03	Reliability	The system should work properly with at least 99% uptime and very few errors (less than 1% failure rate).	System Core Module	High
NFR-04	Security	The system should protect user data by using secure login and encrypted passwords and should only allow access to users with a valid session token.	Login & Authentication Module	High
NFR-05	Scalability	The system should support growth from 100 to 500 users without slowing down more than 20%.	Backend Processing Module	Medium
NFR-06	Availability	The system should be available 24/7 with maximum downtime, not more than 2 hours per month.	Server & Hosting Module	High

NFR-07	Maintainability	The system should allow easy fixing of bugs or updates within 1–2 days.	Admin & Maintenance Module	Medium
NFR-08	Compatibility	The system should work properly on Chrome, Edge, and Android without any major issues.	Frontend Module	High
NFR-09	Data Integrity	The system should keep data accurate with no data loss and less than 1% duplication.	Database Management Module	High
NFR-10	Backup & Recovery	The system should take daily backup and restore data within 1 hour in case of failure.	Database & Backup Module	Medium

2.5 Use Case Analysis

This technique was using case analysis. For each user class, I created detailed use cases describing exactly how the user will interact with the system. Each use case includes a normal flow of events, alternative flows for error conditions, preconditions, and postconditions. This technique is effective for interactive end-user applications like ours because it focuses on what the user wants to accomplish rather than technical implementation details.

2.5.1 Use Case 1

Table 2-16: User Registration

Field	**Description**
Use case id	UC-1
Use case name	User Registration
Actors	User, Business Owner
Description	Allow a new user to create an account on the platform.
Trigger	User selects Register option.
Preconditions	User is not already registered.
Postconditions	Accounts are created successfully.
Normal flow	1. Open Platform 2. Click Register 3. Enter Details 4. Validate Input 5. Save Data 6. Redirect to login
Alternative flow	Emails show errors.

Business rules	Email must be unique.
Assumptions	Internet is available.

2.5.2 Use Case 2

Table 2-17: User Login

Field	Description
Use case id	UC-2
Use case name	User Login
Actors	User
Description	Allow users to log in to the system.
Trigger	User selects Login option.
Preconditions	Users must be registered.
Postconditions	User is logged into the system.
Normal flow	1. Enter email/password. 2. Validate. 3. Grant access.
Alternative flow	Invalid credentials ❼error.
Business rules	Correct credentials required.
Assumptions	User remembers login details.

2.5.3 Use Case 3

Table 2-18: Search Business

Field	Description
Use case id	UC-3
Use case name	Search Businesses
Actors	User
Description	Allow users to search for businesses.
Trigger	Users enter search query.

Preconditions	System is running.
Postconditions	Results are displayed.
Normal flow	1. Enter Keyword 2. Fetch Data 3. Show Results
Alternative flow	No result was found.
Business rules	Results depend on data.
Assumptions	Data exists in databases.

2.5.4 Use Case 4

Table 2-19: View Business Details

Field	**Description**
Use case id	UC-4
Use case name	View Business Details
Actors	User
Description	Shows detailed information about a business.
Trigger	User selects a business.
Preconditions	Business exists.
Postconditions	Details are displayed
Normal flow	1. Select Business 2. Fetch Details 3.Display Info
Alternative flow	Data was not found.
Business rules	Only approved listings shown.
Assumptions	Data is available.

2.5.5 Use Case 5

Table 2-20: Add Business Listing

Field	**Description**
Use case id	UC-5

Use case name	Add Business Listing
Actors	Business Owner
Description	Allow owners to add business.
Trigger	Select Add Business.
Preconditions	User is logged in.
Postconditions	Listing saved as pending.
Normal flow	1. Enter Details 2. Validate 3. Save Data
Alternative flow	Invalid input.
Business rules	Required fields mandatory.
Assumptions	Owner enters correct info.

2.5.6 Use Case 6

Table 2-21: Edit Business Listing

Field	**Description**
Use case id	UC-6
Use case name	Edit Business Listing
Actors	Business Owner
Description	Allow editing business data.
Trigger	Select Edit option.
Preconditions	Listing exists.
Postconditions	Updated data saved.
Normal flow	1. Open listings 2. Edit Data 3. Save Changes
Alternative flow	Invalid update.
Business rules	Only owner can edit.
Assumptions	Owner is authenticated.

2.5.7 Use Case 7

Table 2-22: Delete Business Listing

Field	Description
Use case id	UC-7
Use case name	Delete Business Listing
Actors	Business Owner
Description	Allows deletion of business.
Trigger	Select Delete option.
Preconditions	Listing exists.
Postconditions	Listing removed.
Normal flow	1. Select Delete 2. Confirm 3. Remove Data
Alternative flow	Cancel deletion.
Business rules	Only owner can delete.
Assumptions	User confirms action.

2.5.8 Use Case 8

Table 2-23: Submit Review

Field	Description
Use case id	UC-8
Use case name	Submit Review
Actors	User
Description	Allow users to give reviews.
Trigger	Select Write Review.
Preconditions	User logged in.
Postconditions	Review saved.

Normal flow	1. Enter Review 2. Validate 3. Save
Alternative flow	Invalid review.
Business rules	Rating must be valid.
Assumption	User gives feedback

2.5.9 Use Case 9

Table 2-24: View Reviews

Field	**Description**
Use case id	UC-9
Use case name	View Reviews
Actors	User
Description	Displays reviews of a business.
Trigger	Open business page.
Preconditions	Reviews exist.
Postconditions	Reviews shown.
Normal flow	1. Open page 2. Fetch reviews 3. Display
Alternative flow	No reviews.
Business rules	Only valid reviews shown.
Assumptions	Data exists.

2.5.10 Use Case 10

Table 2-25: Admin Approval

Field	**Description**
Use case id	UC-10

Use case name	Admin Approval
Actors	Admin
Description	Admin verifies listings.
Trigger	Open pending list.
Preconditions	Admin logged in.
Postconditions	Status updated.
Normal flow	1. View listings 2. Approve/reject 3. Save
Alternative flow	Reject listing.
Business rules	Only admin can approve.
Assumptions	Admin reviews carefully.

2.5.11 Use Case 11

Table 2-26: Notification System

Field	**Description**
Use case id	UC-11
Use case name	Notification System
Actors	System.
Description	Send notifications to users.
Trigger	Event occurs (approval etc.).
Preconditions	System is active.
Postconditions	Notification sent.
Normal flow	1. Event triggered. 2. Generate Message 3. Send notification
Alternative flow	Failed delivery.
Business rules	Notifications must be relevant.

Assumptions	Email service working.

2.5.12 Use Case 12

Table 2-27: Location Services

Field	Description
Use case id	UC-12
Use case name	Location Services
Actors	User
Description	Show business locations on map.
Trigger	Open map/search.
Preconditions	Internet + API available.
Postconditions	Locations displayed.
Normal flow	1. Fetch coordinates 2. Load map 3. Show markers
Alternative flow	Map not loading.
Business rules	Use accurate location data.
Assumptions	Mapping service (OpenStreetMap) is available.

2.5.13 Use Case Diagram

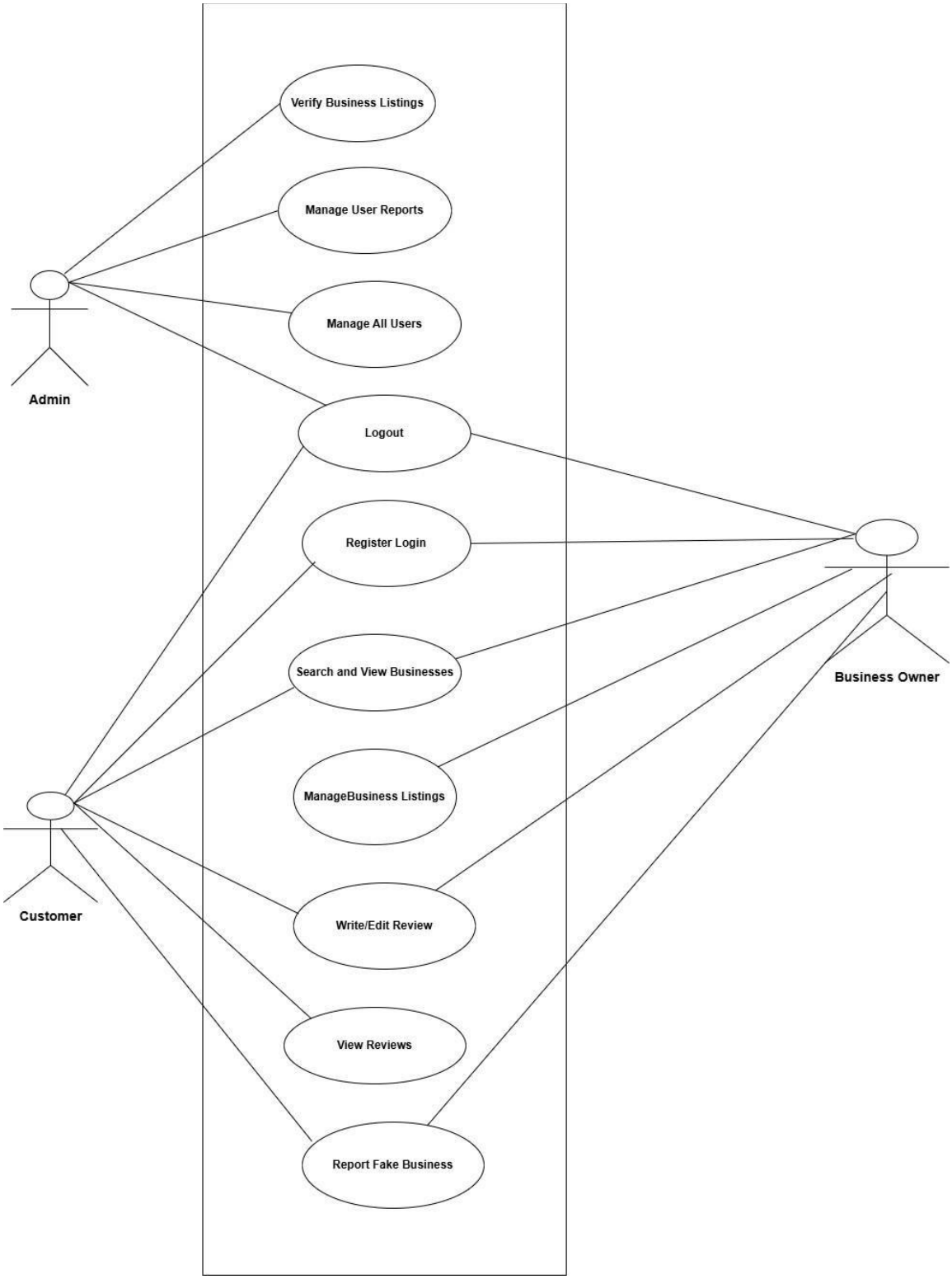


Figure 2-1: Use Case Diagram

2.6 External Interface Requirements

This section describes how the system will communicate with external elements such as users, other software, hardware, and networks.

2.6.1 User Interface Requirements

- The system is web-based and works on all modern browsers like Chrome, Firefox and Edge.
- The interface is responsive, which means it automatically adjusts to mobile phones, tablets, and desktop computers.
- On desktop computers, the screen shows multiple columns. On mobile phones, it shows one column with bigger buttons for easy touch.
- All text fonts are at least 14 pixels so users can read comfortably.
- The color scheme has good contrast between text and background.
- Buttons change color when the mouse hovers over them so users know they can click.
- A loading spinner appears whenever the system is fetching data, so users know how to wait.

2.6.2 Software Interface

Our system connects two external software components:

- OpenStreetMap, through the Leaflet library and the Nominatim geocoding service - shows the interactive map, places business pins, and converts a business address into map coordinates.
- SMTP Email Service (Nodemailer with Gmail) - this service sends email notifications to business owners when their listing is approved or rejected, alerts the admin when a new listing or report is submitted, and sends password-reset links.

No other external software is needed for this system.

2.6.3 Hardware Interface

Since this is a web-based application that runs on a server, there are no direct hardware interfaces. Users access the system through their device's web browser on mobile phones, laptops or desktop computers. No special hardware is required.

2.6.4 Communication Interface

- The front end and backend communicate over HTTP during development; in a production deployment this would run over HTTPS to keep data secure.

- All communication happens in JSON format, which is lightweight and easy to read.
- The React.js frontend sends REST API calls to the Node.js backend.
- Example: when a user searches for a restaurant, the frontend sends a GET request to /Api/businesses? Search=restaurant.
- The backend then returns a JSON array containing all matching businesses.
- Map tiles are also loaded over HTTPS, from OpenStreetMap's servers.

2.7 Summary

Summary of Requirements Identification and Use Case Analysis for Local Business Support Platform:

1. Requirements Identification Process

key processes involved are:

Stakeholders include normal users, business owners, and admin.

Both functional requirements (like search, reviews, business listing) and non-functional requirements (like speed, security, easy use) are collected. methods like use cases are used to clearly understand system needs.

2. Use Case Analysis Process

The process includes:

Identifying Actors: Find all users of the system such as users, business owners, and admin.

Developing Use Cases: Make simple cases like searching a business, adding a business, and managing the system.

3. Importance

Here's why this process is important:

- It helps everyone understand what the system will do and how it will work.
- It makes system design easy and helps build a simple and useful platform.

Chapter 3: Design and Architecture

3. System Design

This chapter describes how we built our Local Business Support Platform. We describe the overall structure, key components, data storage and how users will interact with the system. The design is based on three-tier architecture (frontend, backend and database) so that the system is fast, secure and easy to maintain.

Product perspective

The Local Business Support Platform is a web-based application that connects sers, business owners and admins in one place. The system enables users to search for local business, business details, reviews and see locations on a map. Business owners have the possibility to add and manage their listing, and the admins are verifying the information to keep the platform authentic.

The product acts as an intermediating layer between:

- Users who need reliable local business details.
- Businesses on the other hand want an easy way to get their business in front of potential customers online.
- Admins who keep the system safe and verified.

System Overview

Our system is a web-based system having three major modules:

1. **Customer Module -** Allow users to search, filter, see businesses on map, post reviews and report fake listings.
2. **Business Owner Modul -** sign us as an owner, create/delete/edit listings, post special offers and read customer feedback.
3. **Admin Module -** login to a secure dashboard, approve or reject pending business listings, manage user accounts and check on reporting listings.

The system follows a three-tier architecture:

- **Frontend (React.js) -** Handles all user interfaces and sends API requests.

- **Backend (Node.js + Express.js) -** Handles the request, apply the business logic and communicate with database.
- **Database (MySQL) -** Stores all information of the structured data: users, businesses, reviews, reports and promotions.

External services like OpenStreetMap are integrated into the system for location display and geocoding.

3.1 Design Considerations

Assumption and dependencies:

- We assume that all users (customers, business owner and admin) have a stable internet connection because our platform is web based.
- Business owners must maintain accurate and up to date listing information.
- The system depends on OpenStreetMap's Nominatim service to convert addresses to coordinates. If this service is unreachable, new businesses are still saved, but without a map pin until the address is corrected or the service is available again.
- We also depend on email notification service (like Gmail SMTP) to inform business owners when their listings is approved or rejected.

Limitations:

- If a business owner does not update their information, users may see outdated addresses or timings.
- OpenStreetMap's Nominatim service asks that free usage stay under about one request per second. If our platform becomes very popular, we may need to run our own Nominatim instance or use a paid geocoding provider.
- Our platform does not handle payments or deliveries - those are managed directly between customers and businesses.
- The system is designed only for local businesses in our area (Gujranwala). It does not cover other cities.

Risks:

- **Fake or Spam Listings:** Unverified business owners could add false or misleading information. We solved this by adding an Admin verification step - every new listing must be approved before the admin before it gets published.
- **Data Security:** User passwords and business data could be at risk. We will encrypt passwords and use secure authentication (JWT tokens).
- **Slow performance under heavy traffic:** when many people search at once, the system might slow down. In the future, we plan to add cash to improve speed.

3.2 Design Models

We used object-oriented design to model our system. The main models are the class diagram, sequence diagram and a state transition diagram.

1. **Class Diagram**

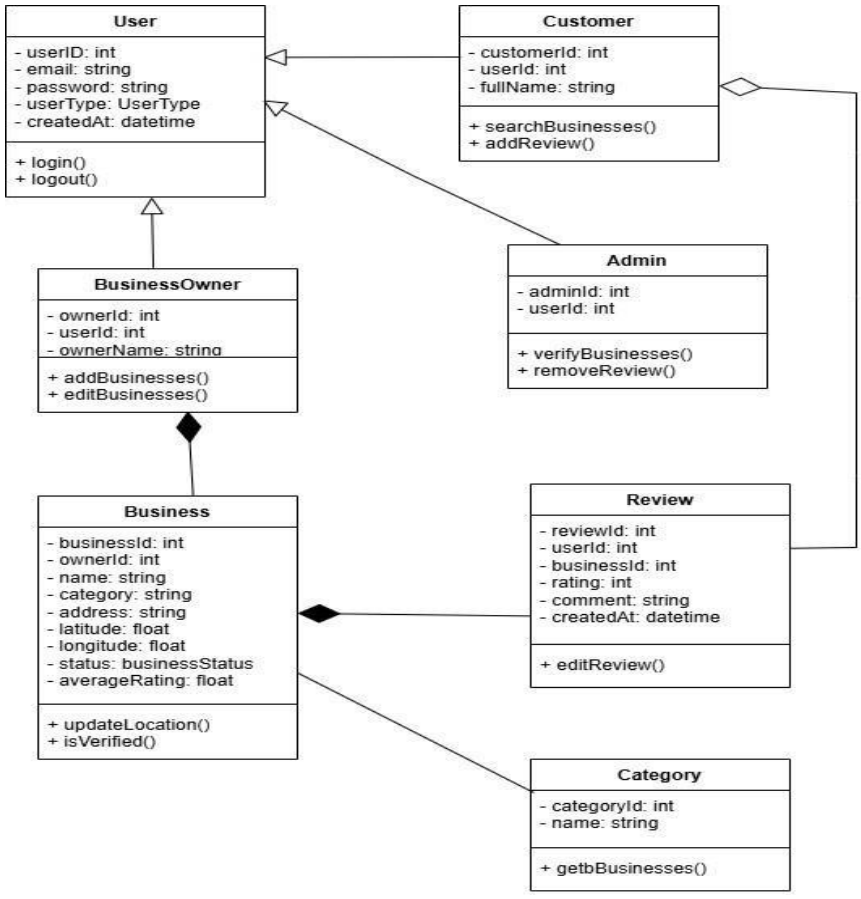


Figure 3-1: Class Diagram

2. **Sequence Diagram**

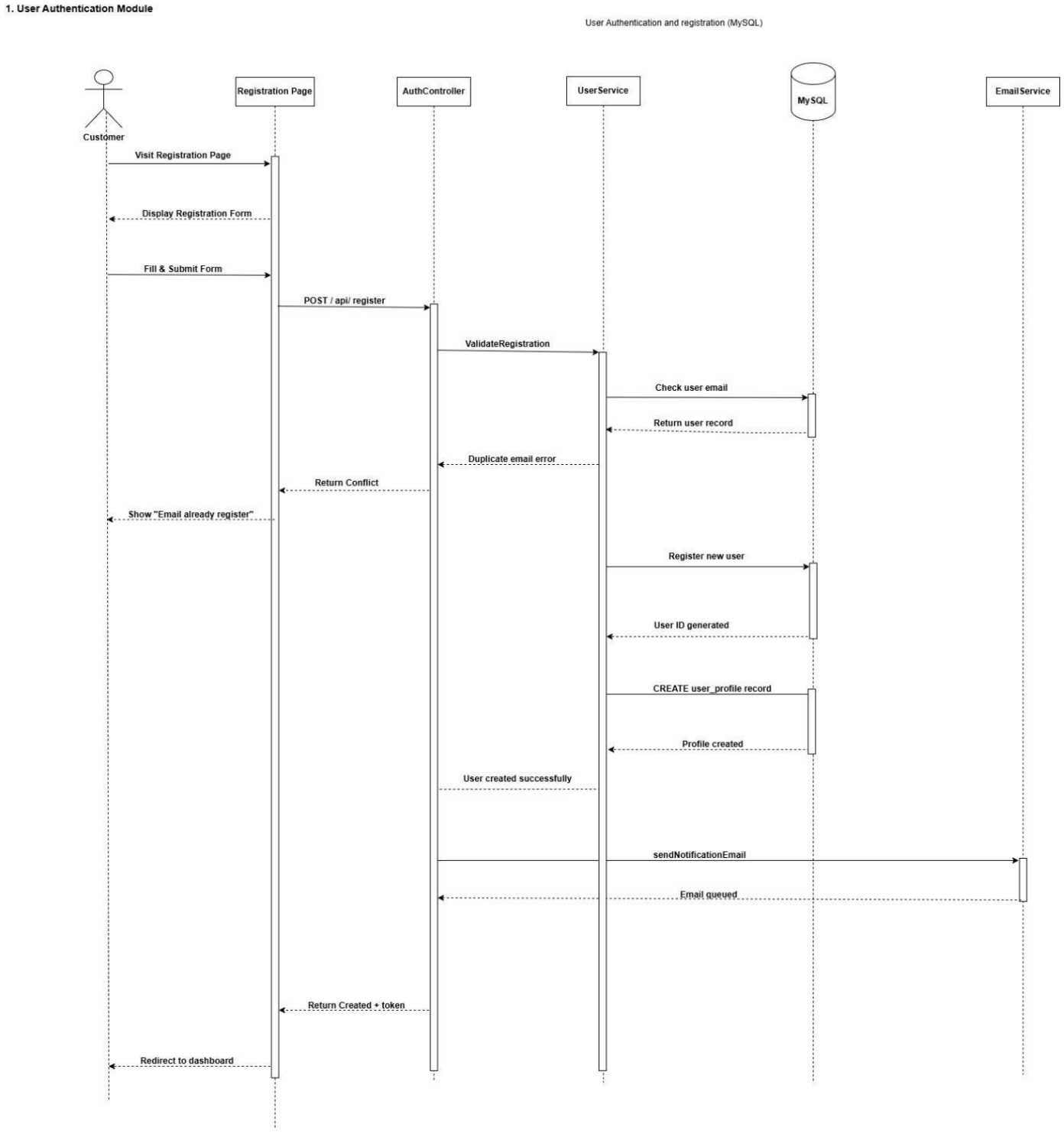


Figure 3-2: Sequence Diagram 1

2.Business Registration Module

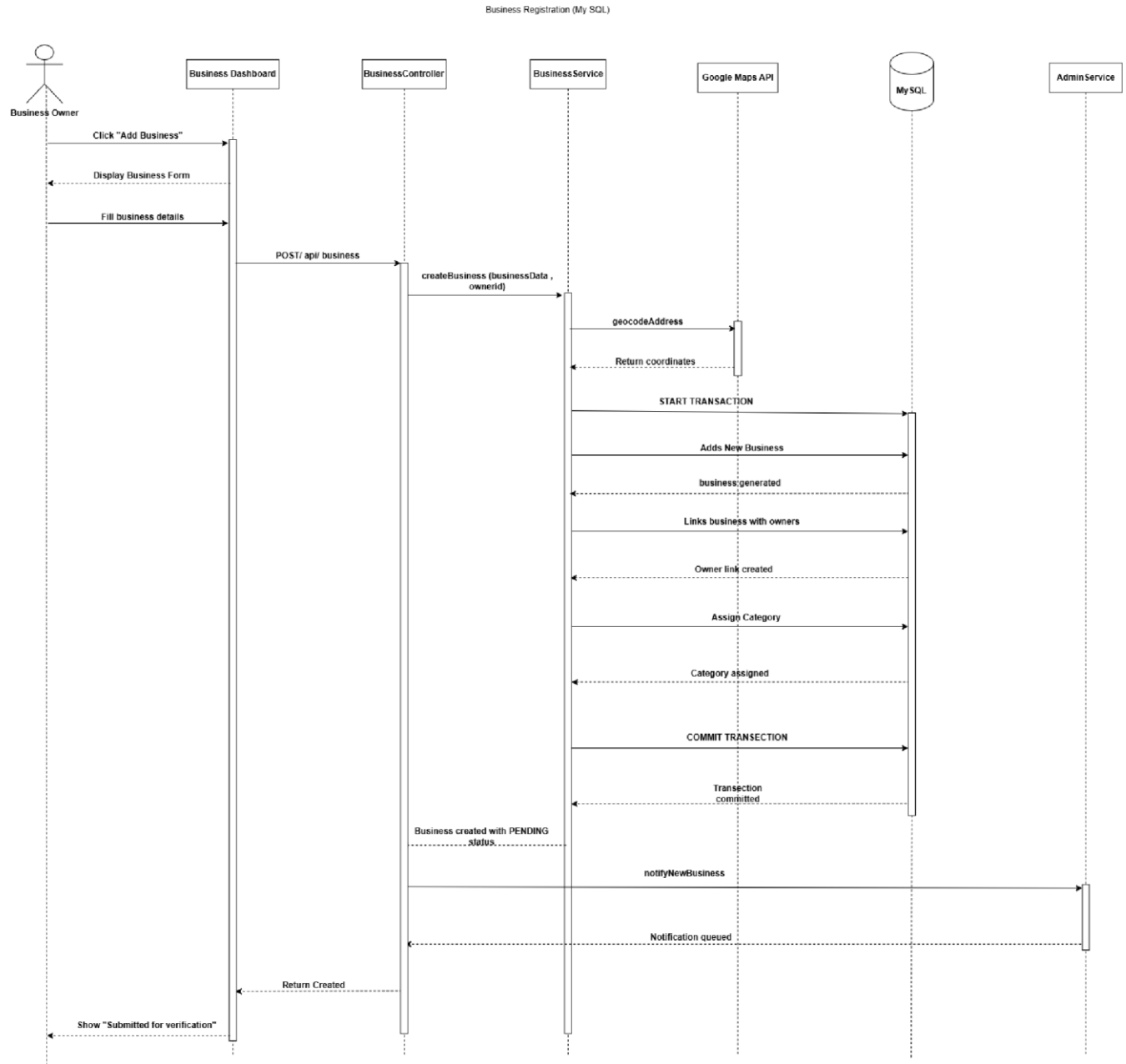


Figure 3-3: Sequence Diagram 2

3. BUSINESS SEARCH MODULE

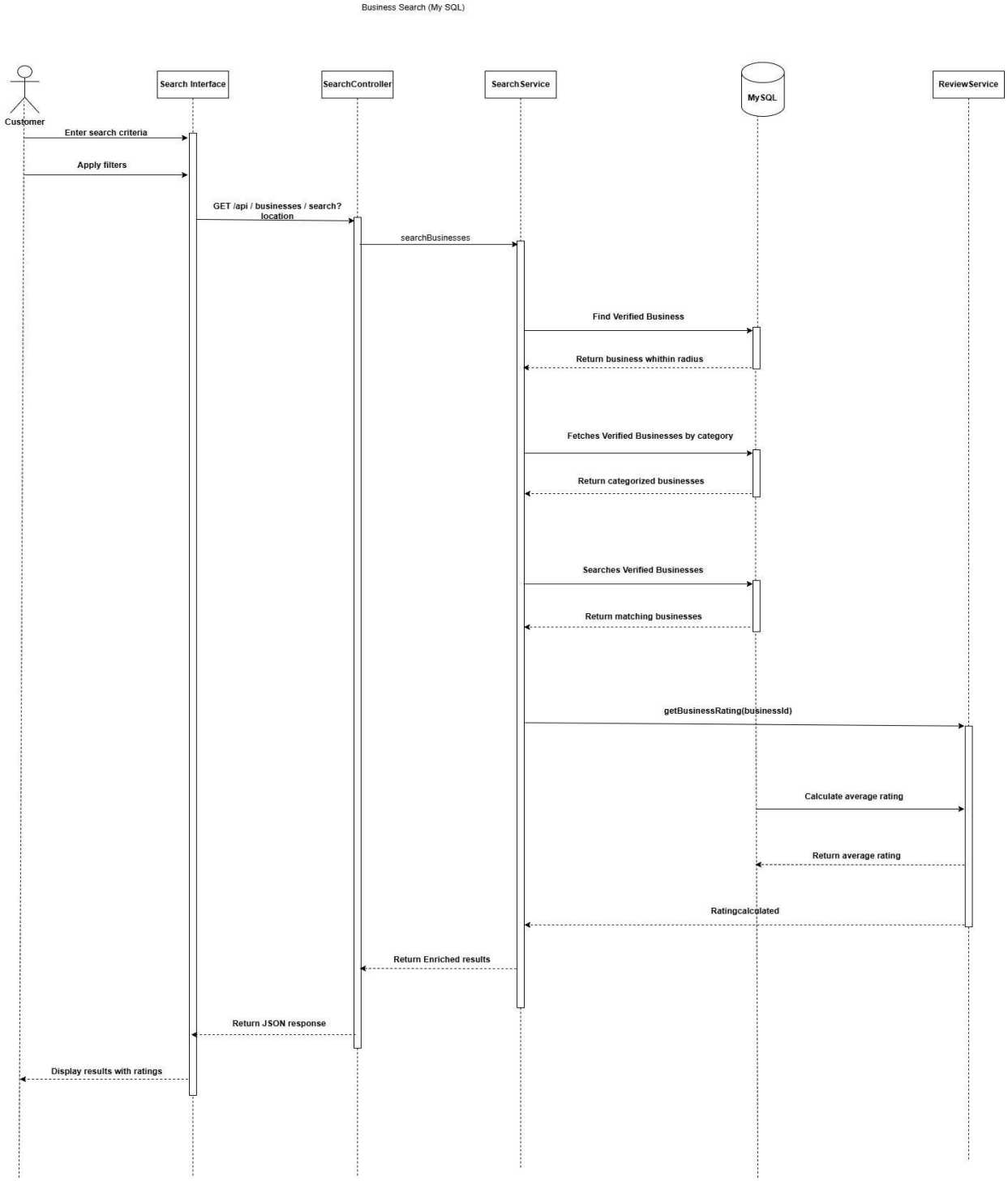


Figure 3-4: Sequence Diagram 3

4. REVIEW SYSTEM MODULE

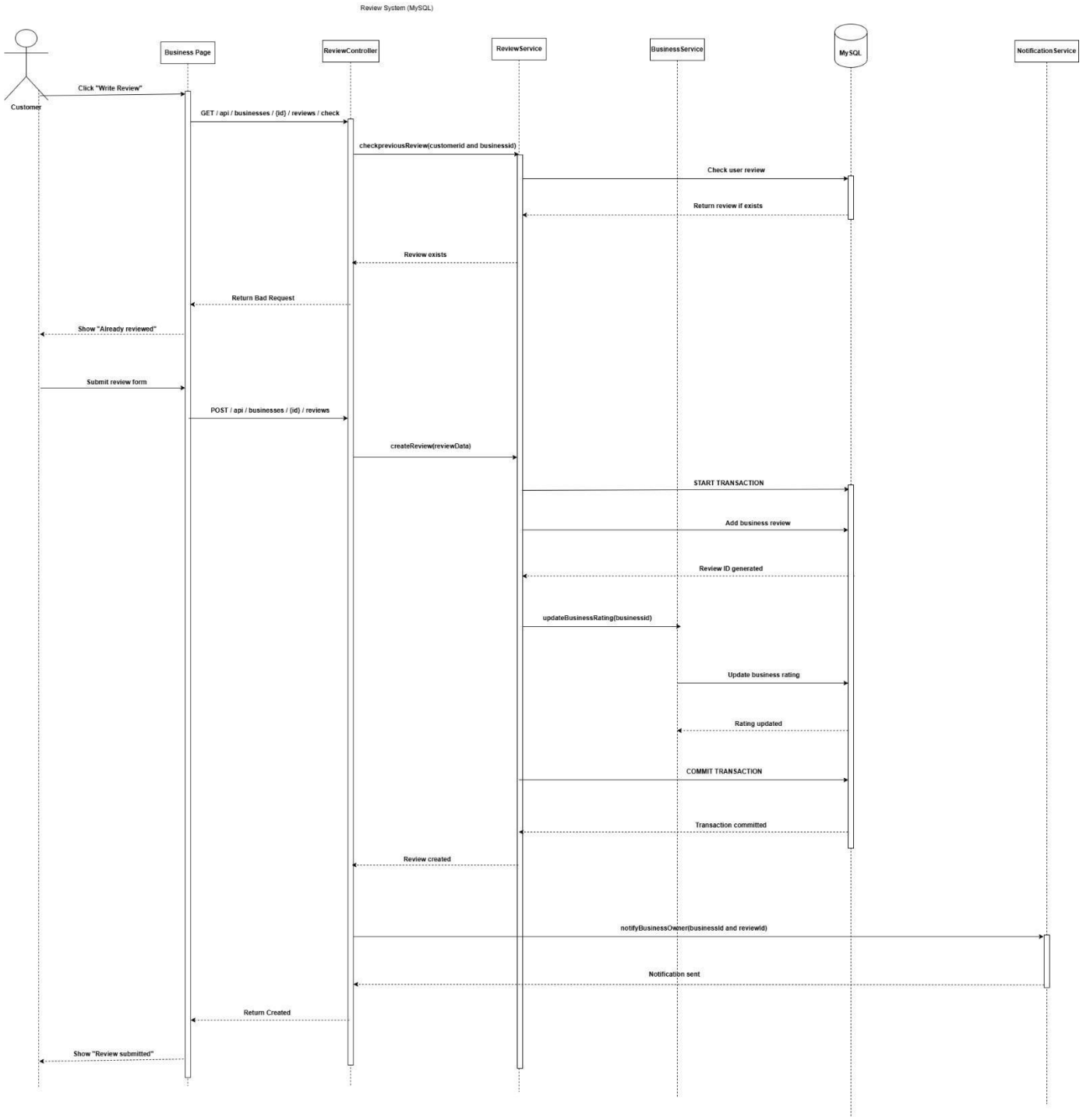


Figure 3-5: Sequence Diagram 4

Admin Dashboard Statistics Flow

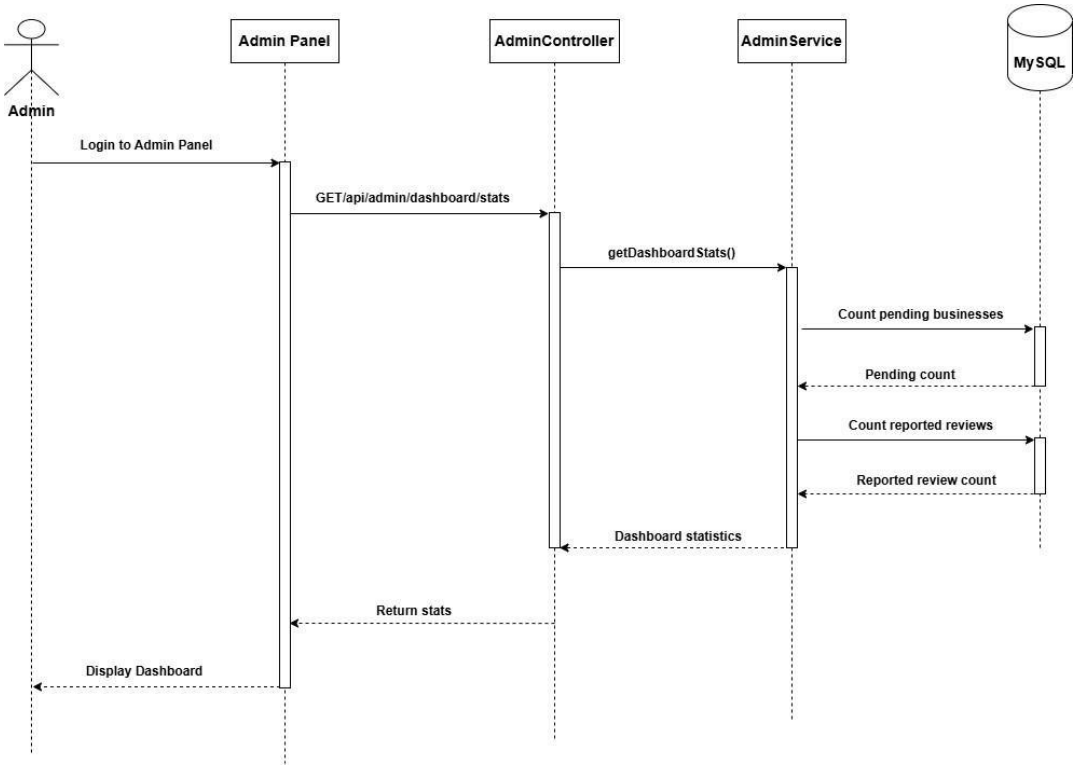


Figure 3-6: View Pending Businesses

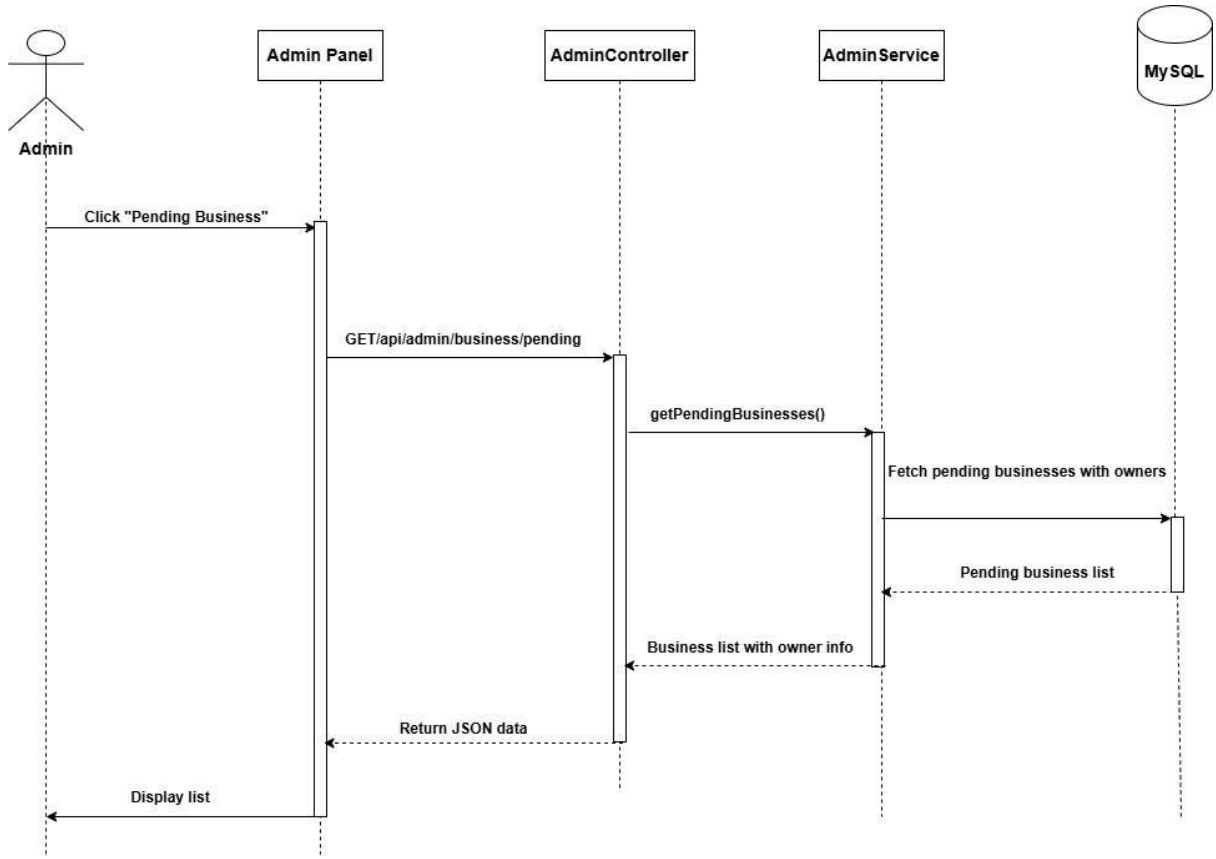


Figure 3-7: Verify Business Process

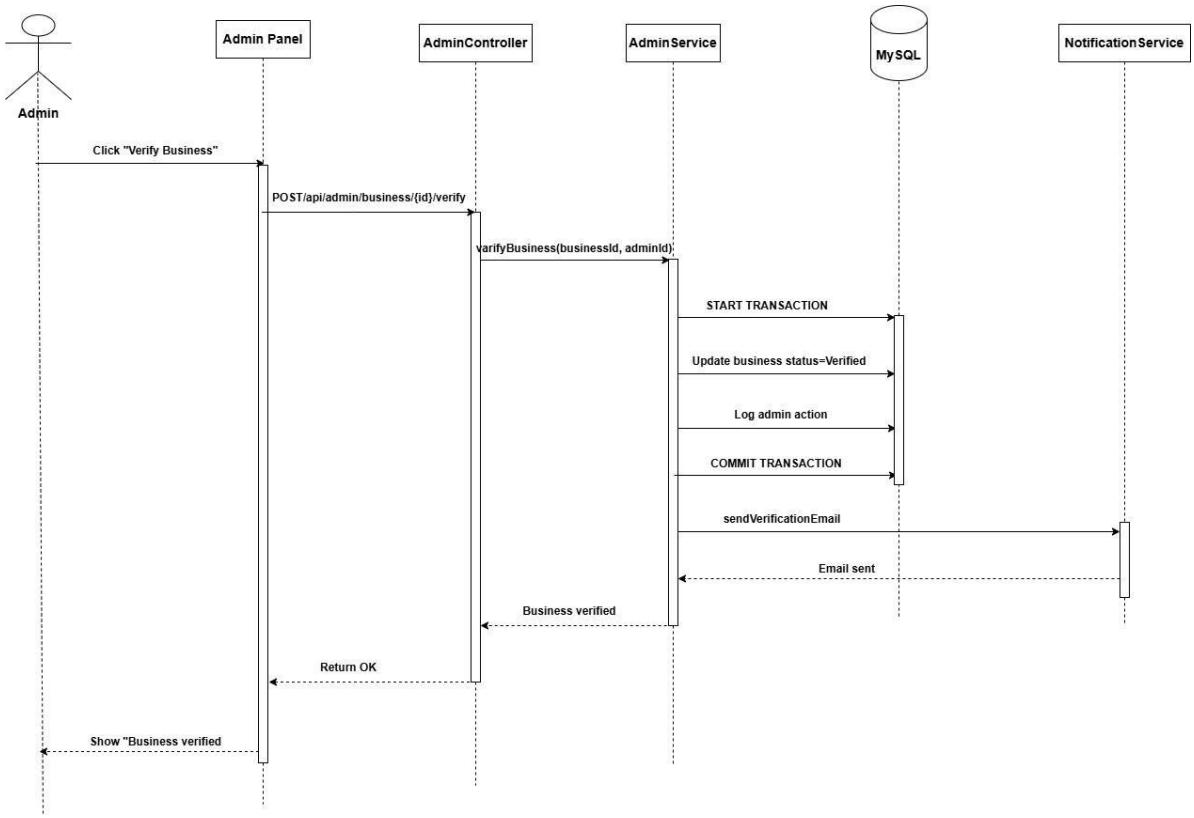


Figure 3-8: Sequence Diagram 5

6. BUSINESS OWNER DASHBOARD

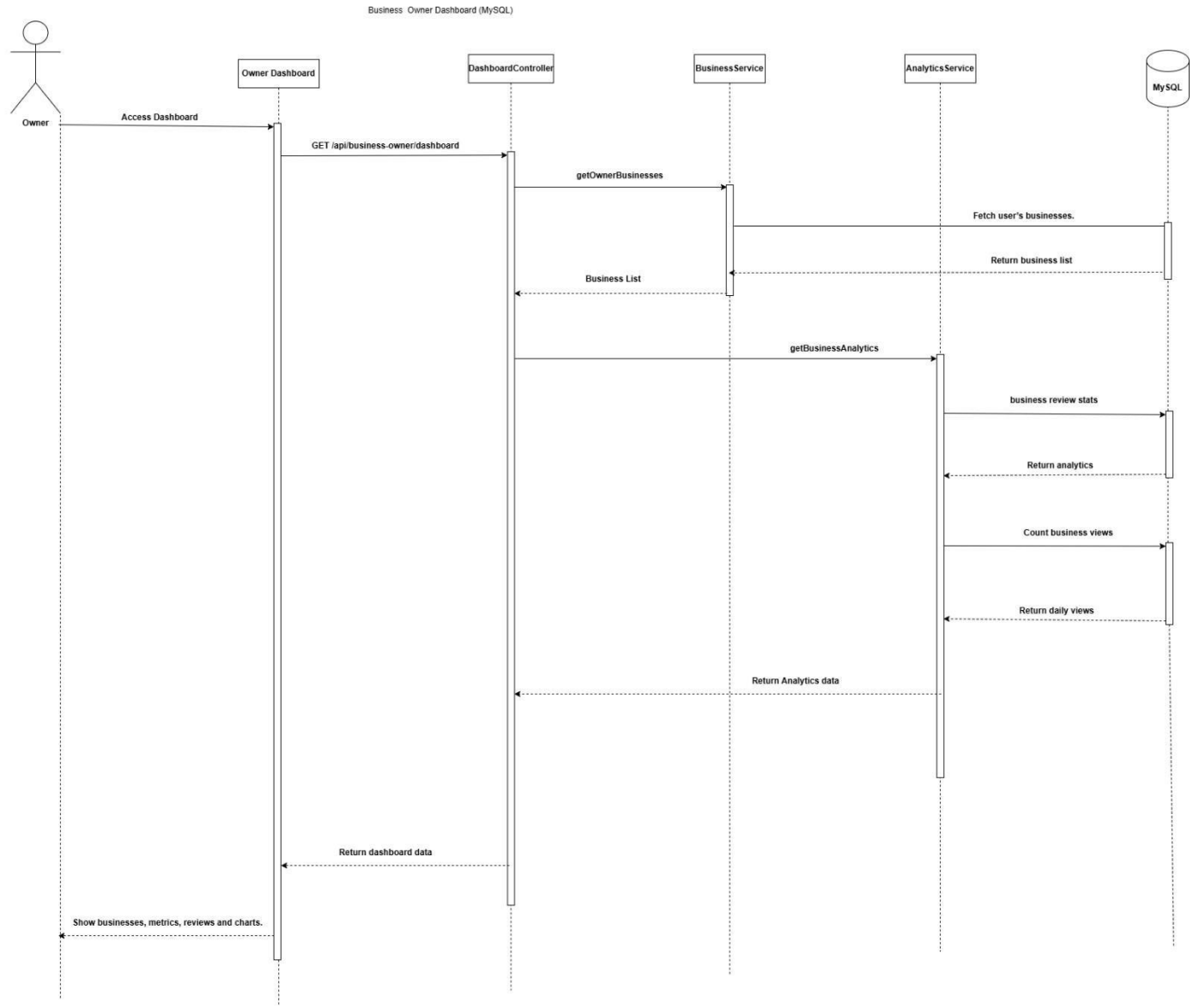


Figure 3-9: Sequence Diagram 6

7. NOTIFICATION SYSTEM

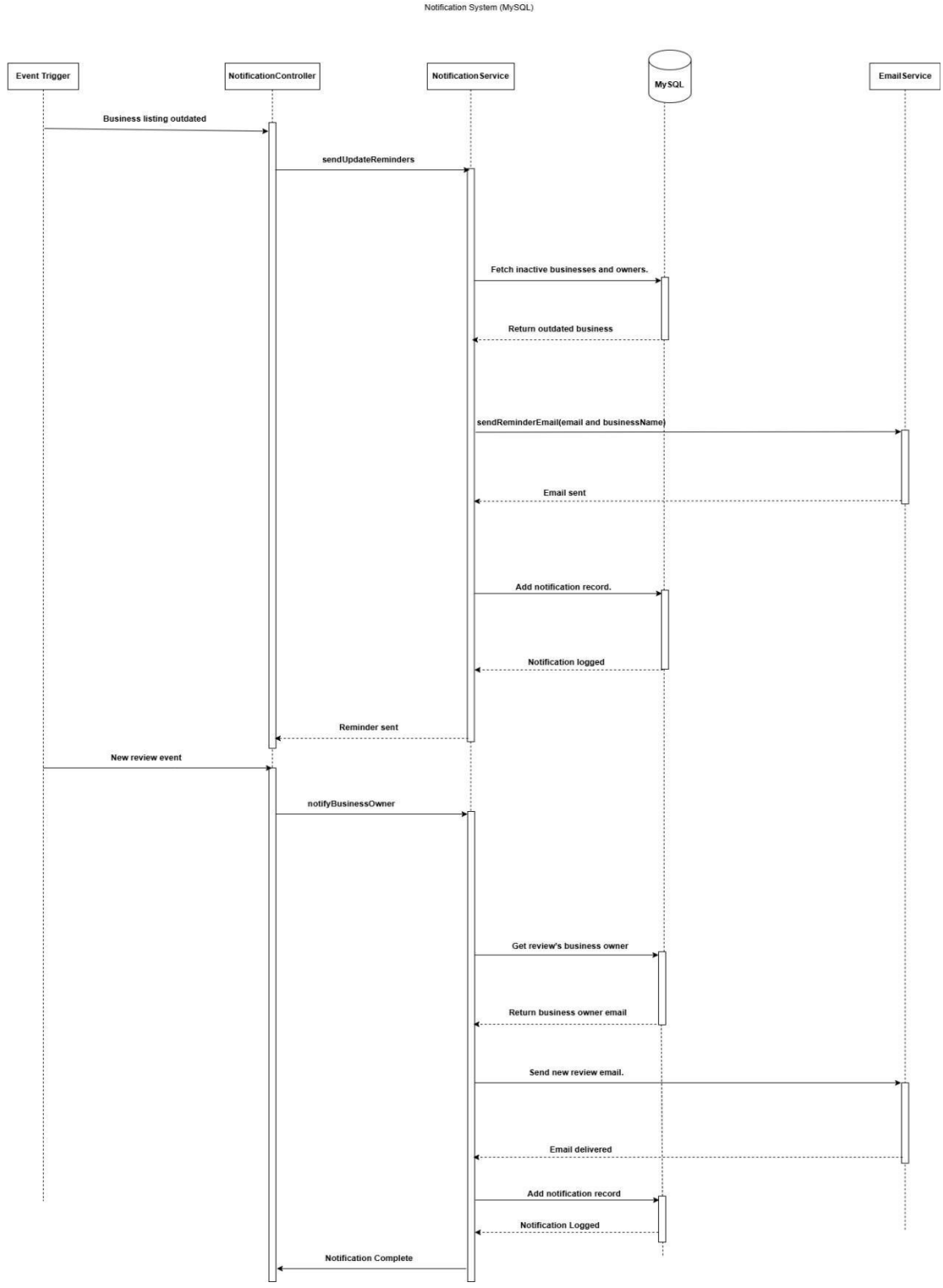


Figure 3-10: Sequence Diagram 7

8. Location Service

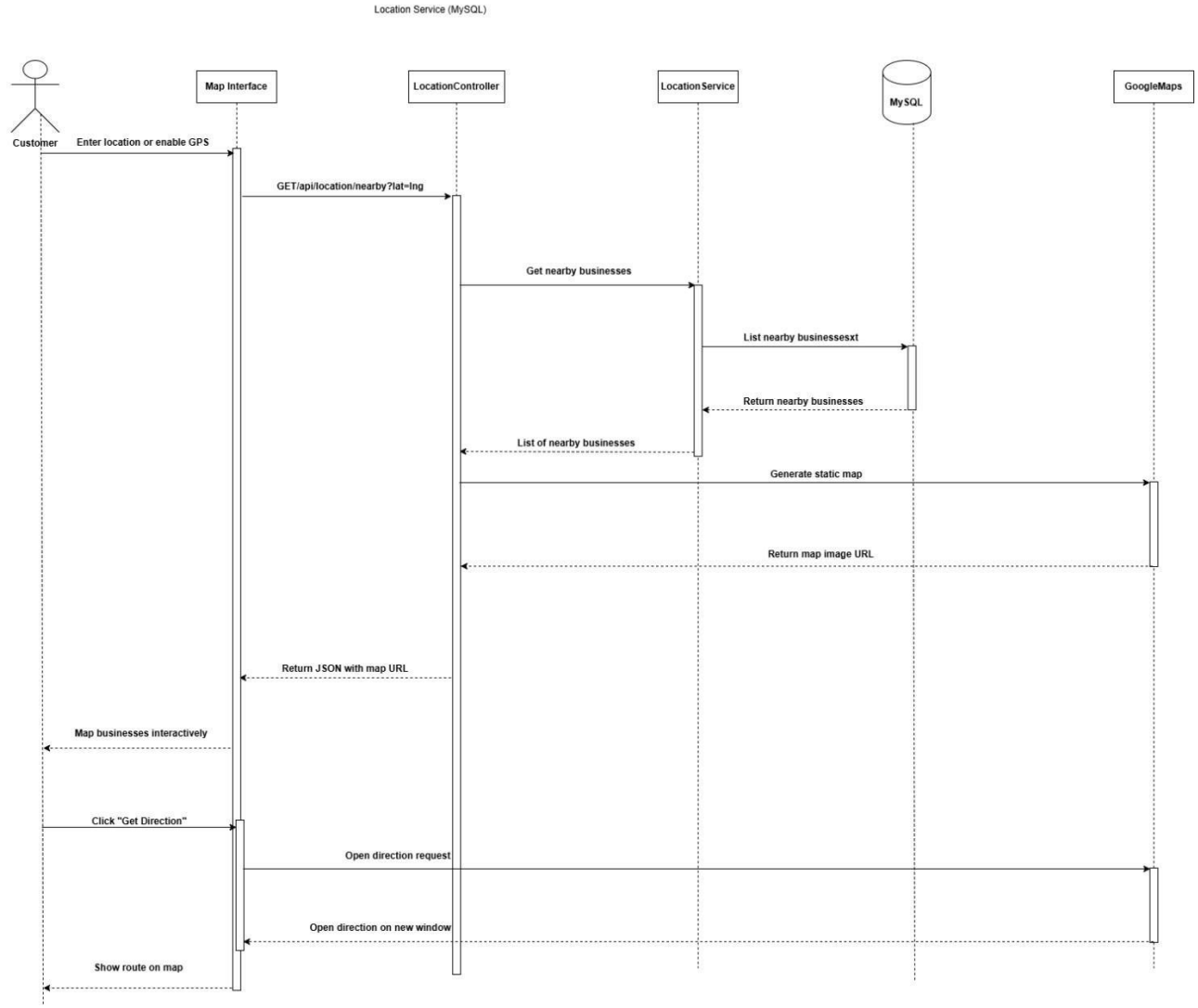


Figure 3-11: Sequence Diagram 8

3. State Transition Diagram

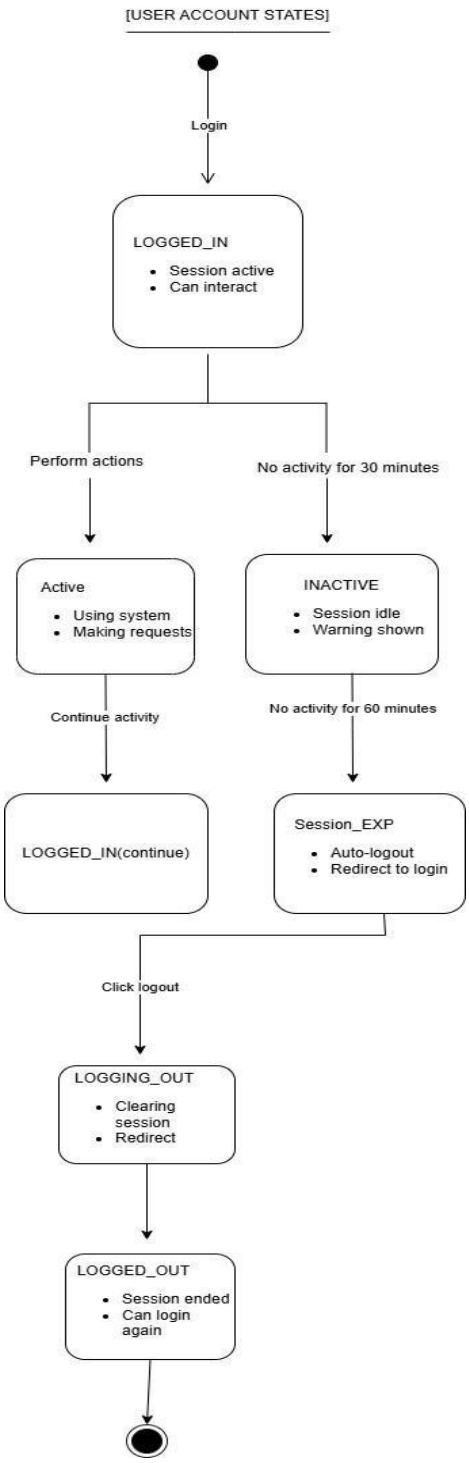


Figure 3-12: State Transition Diagram 1

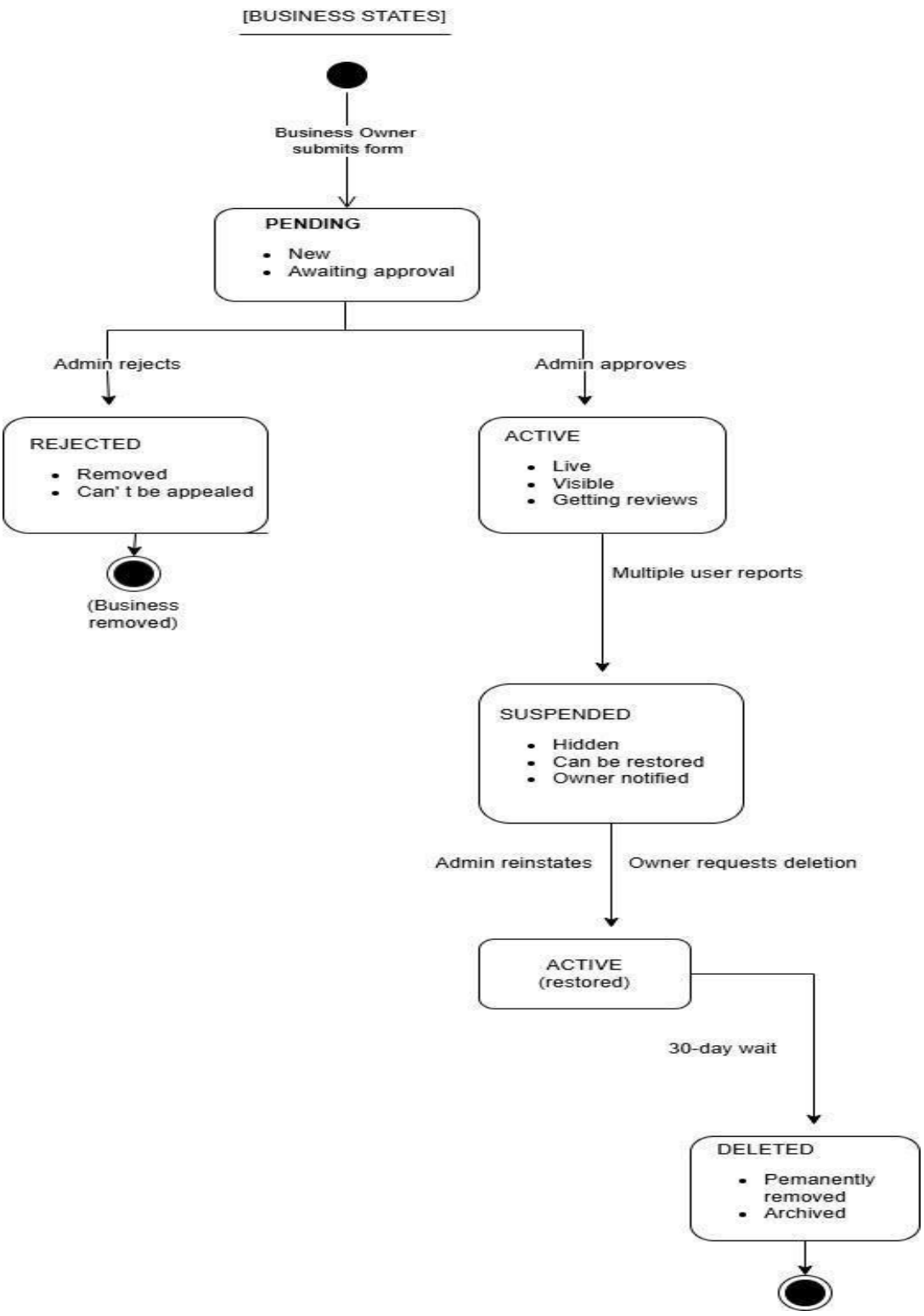


Figure 3-13: State Transition Diagram 2

3.3 Architectural Design

Our system follows a three-tier architecture. This keeps the user interface, business logic and data storage separate, making the system easier to maintain and scale.

The three tiers are:

- **Presentation Tier (Frontend)** - Built with React.js. It runs in the user's browser and includes pages for customers, business owners and the admin panel.
- **Logic Tier (Backend)** - Built with Node.js and Express.js. It provides RESTful APIs, handles authentication, processes search requests, manages reviews and applies business rules.
- **Data Tier (Database)** - MySQL database. It stores all users, businesses, reviews, reports and promotions.

External services:

- OpenStreetMap (via Leaflet) - used to display business locations and convert addresses to coordinates.
- Email service - sends notifications about listing approval or rejection.

1. UML Class relationship diagram

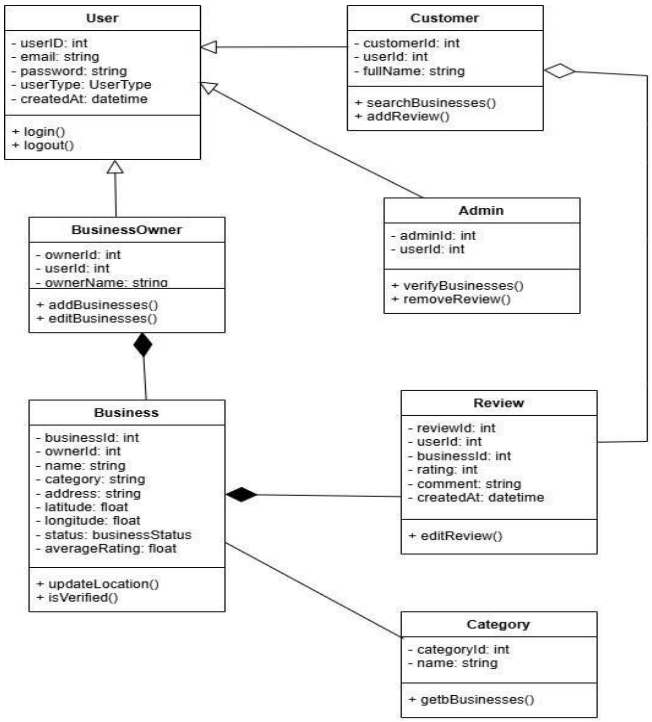


Figure 3-14: UML Class Relationship Diagram

2. UML Component diagram



Figure 3-15: UML Component Diagram

3. Model View Controller (MVC)

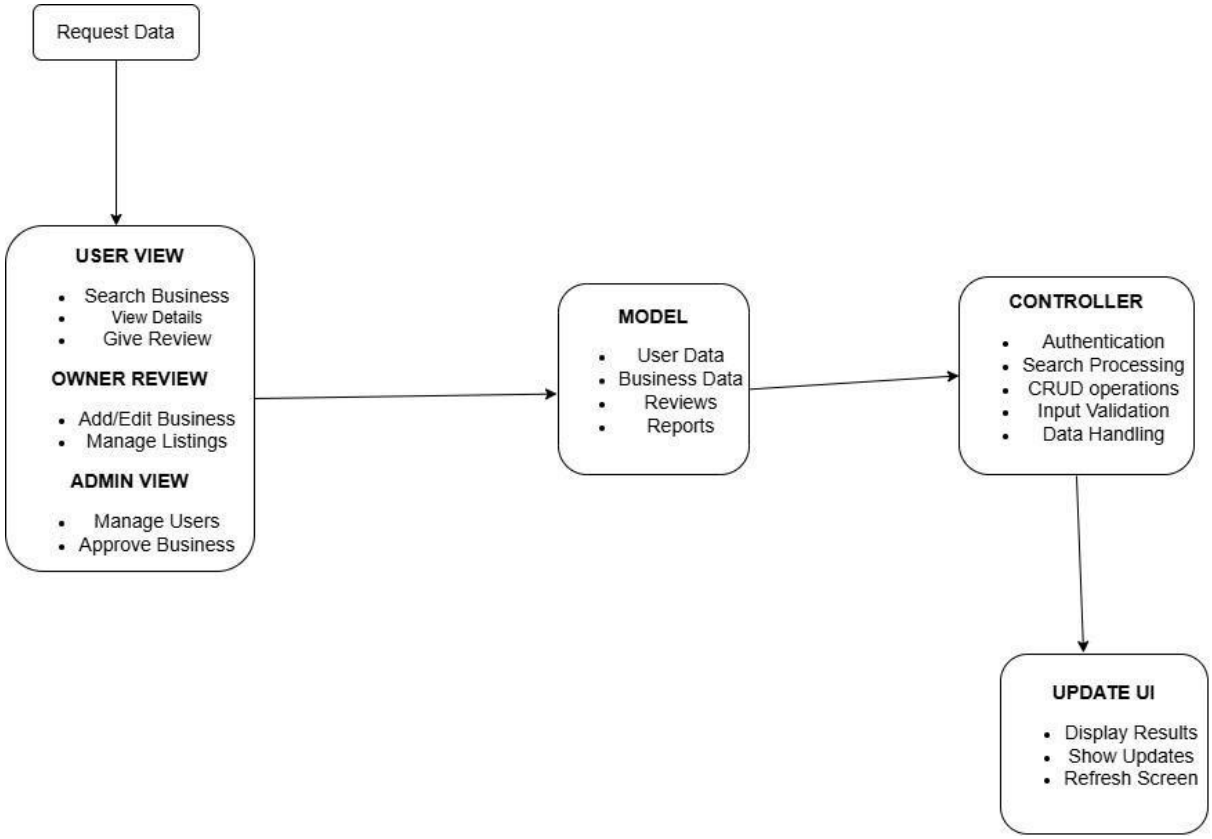


Figure 3-16: Model View Control Diagram

4. Mapping to Architecture Pattern

We did not just build random components - we intentionally design our system along the lines of established architecture styles. We used layered architecture (three-tier) combined with the Model-View-Controller (MVC) on the server side. Below is how each part maps these patterns.

4.1 Layered (Three-tier) Pattern Mapping

Table 3-1: Layered (Three-tier) Pattern Mapping

Tier	**What it contains in our project**	**Responsibility**
Presentation Tier	React pages: Landing page, Search page, Business Details page, Dashboard (customer/owner/admin)	Display data to the user, capture input and send API requests. No business logic here.
Logic Tier (Application Tier)	Node.js + Express: API routes (e.g., /Api/business/add, /Api. Search), Controller, authentication middleware	Apply business rules (e.g., only admin can approve listings), validate data and interface between frontend and database.
Data Tier	MySQL database: tables for users, businesses, reviews, reports, promotions	Store and retrieve persistent data. Ensure data integrity using foreign keys and constraints.

Why does this mapping work for us:

Because each tier has one and only one reason to change. For example, if we later want to change the frontend framework (say from React to Vue), we only touch the presentation tier - the backend and database remain unchanged. Similarly, if we need to optimize database queries, we work only on the data tier. This separation also improves security because the frontend never directly accesses the database.

4.2 MVC (Model-View-Controller) Pattern Mapping (on the backed) Even within our Node.js backend, we use MVC to keep things organized.

Table 3-2: MVC (Model-View-Controller) Pattern Mapping (on the backend)

MVC Component	How we implement it in our project
Model	MySQL tables+ SQL queries (or ORM like Sequelize). The business logic module contains the code that handles the business model functions like insertion, removal and search etc.
View	In a traditional MVC, the view is the UI. But in our REST API backend, the "view" is the JSON response sent to the frontend. For example, rest. Json (business: list) is the view.
Controller	Express route handler. For example, businessController.js contains add business (), getallbusiness (), approve business () etc. Each controller receives the requests, calls the model and sends the JSON response

3.4 Data Design

Data Design tells us **how information is stored and organized** within the database.

Our system is based on a relational database, where data is stored on tables and connected using IDs.

The Key Entities of Our System:

- Users
- Business Listings
- Reviews
- Promotions
- Reports

Every entity is described by various data fields (attributes).

Information Domain Transformation

In the Local Business Support Platform, the raw information collected from users, business owners, and admins is converted into structured data that can be stored, processed and retrieved efficiently. Information Sources:

- User Input (Search terms, reviews, reports)
- Business owner submissions (business details, promotions)
- Admin decisions (approval, rejection)

Transformation Process:

1. **User Input →Validated Data**

   Search for keywords, reviews, and reports are validated and sanitized.

2. **Business Owner Inputs → Business Object**

   The business information becomes a Business Listing object

3. **Admin Actions → Updated Status**

   Approval/rejection changes business status field in database.

4. **Location Data → Coordinates**

   Using OpenStreetMap's Nominatim service, addresses are mapped to lat/long.

   The system converts all unorganized information into well-defined objects and relational tables.

Data Structure

The major data structures used in the system are:

Relational Tables (MySQL)

These tables contain:

- Users
- Businesses
- Reviews

- Reports
- Promotions
- Admin Decisions

Object Models (OOP Classes)

Classes such as User, Businessowner, Business Listing map directly to database tables.

JSON Data Objects

Used for:

- API communication between frontend (React) and backend (Node.js/Express)
- Sending review data
- Sending search result list
- Updating business details

Relationships

1. **User -Review (One-to-Many)**

   One user can write many reviews, but each review belongs to only one user.

2. **User -Report (One-to-Many)**

   One user can report multiple businesses.

3. **Businessowner -Business Listing (One-to-Many)**

   One business owner can have many listings.

4. **Business Listing -Review (One-to-Many)**

   A Business can have many reviews from different users.

5. Business Listing -Reports (One-to-Many)

Business can be reported multiple times.

Dataset/Database Design

This section defines how the data is grouped into datasets to enable high-performance storage and retrieval.

1. User Dataset

Stores all user-related data such as login, profile, and role.

2.Business Dataset

Stores:

- Business details
- Address
- Category
- Description
- Status

3.Dataset Review

Stores:

- Ratings
- Review text
- User ID
- Business ID

4.Report Dataset

Stores:

- Report from user
- Reasons
- Admin review status

5. Promotion Dataset Stores:

- Promotion ID
- Business ID
- Promotion text

Normalization Rules:

Normalization is used to remove duplication of data and to keep the database clean and efficient.

How Normalization is implemented in our System:

1. First Normal Form (1NF): First normal form is applied when each cell of a table should hold a single value, and each record needs to be unique.

Example:

Each Business has one category.

1. **Second Normal Form (2NF):** All non-key attributes must depend on the primary key.

Example:

The review text and the rating in the Review table depend only on review ID not on user ID or business ID separately.

2. **Third Normal Form (3NF):** There should be no transitive dependencies.

Example:

Business owner information is stored on the user table, not repeated in the Business table.

3.4.1 Data Dictionary

The Data Dictionary contains all the important data elements used in the system. It defines each item clearly so developers can understand its purpose and how it is used during system operations.

Table 3-3: Alphabetical Two-Column Format (Terminology & Descriptions)

Terminology (Object, Attributes, Parameter)	**Description (Data Type + Purpose)**
Address	String-Physical location of the business.
Admin ID	Integer-Unique ID for admin accounts.
Business ID	Integer-Unique identifier for each business listing.
Business Name	String-Name of the business.
Category	String-Type of business (Restaurant, Salon, Shop).
Coordination's	String (lat, long) Used for map pin location.
Description	String-Short introduction or business summary.
Email	String- Login identifier for user/business owner/admin.
Owner ID	Integer-ID of the business owner.
Password	String (Encrypted) Used for secure login.
User ID	Integer-Unique ID for each account.
City	String - the city the business is in (e.g. Lahore, Gujranwala, Karachi).
Phone	String - contact number of the business.
Hours	String - the business's opening hours.

Status	Enum (pending, active, rejected, suspended) - the current approval status of a business.
Is Verified	Boolean - whether the admin has verified the business.
Photo URL	String - file path to the business's uploaded photo.
Average Rating	Decimal - calculated automatically from all reviews left for a business.
Review ID	Integer - unique identifier for a review.
Rating	Integer (1-5) - the star rating given in a single review.
Comment	String - the written text of a review.

Object-Oriented (OO) Approach: Objects, Attributes, Methods. & Method Parameters:

Here we describe the system using object-oriented structure.

1.Object: User Attributes:

- User ID
- Username
- Email
- Password
- User type

Methods:

- Login ()
- Search Business (keyword)
- View business (business ID)
- Write Review (business ID, rating, review Text)
- Report Business (business ID, Report Reason)

2.Object: Business Owner

Attributes:

- Owner ID
- Full NAME
- Email
- Phone Number Methods:

Methods:

- Add Listing ()
- Update Listing (business ID)
- Add Promotion (business ID, promotion Text)
- View Reviews (business ID)

3. Object: Business Listing

Attributes:

- Business ID
- Owner ID
- Business Name
- Category
- Address
- Coordinates
- Description
- Status

Methods:

- Display Details ()
- Update Business
- Calculating Rating

4.Object: Review

Attributes:

- Review ID
- User ID
- Business ID
- Rating
- Review Text
- Review Status

Methods:

- Submit Review ()
- Approve Review ()
- Delete Review ()

5. Object: Report:

Attributes

- Report ID
- User ID
- Business ID
- Report Reason
- Status

Methods:

- Submit Report ()
- Review Report ()
- Close Report ()

3.5 User Interface Design

This section describes how the system looks and works from the user's point of view. It describes the role of different users, the main features they can use and the feedback messages the system shows. The UI is designed to be simple, clean and easy for everyone normal users, business owners and admins.

User role in the interface

Our system has three types of users:

1. Normal user

- Search nearby businesses
- View business details on the map
- Read and Write Reviews

2. Business owner

- Add new business listings
- Edit existing business details
- Add promotions
- Respond to reviews

3. Admin

- Approve or reject business listings
- Manage users and listings

Feedback information

The system shows clear messages when performing actions:

- Business added successfully
- Your review has been submitted
- Listing is pending approval

- Invalid login detail
- Report submitted

These messages help the user understand the result of their actions.

Expected features

These are the features every user expects when using the system:

- Fast business searching
- Simple navigation
- Clear dashboard for the business owner
- Accurate map location

Simple reviewing and reporting system

3.5.1 Screen Images

Landing Page (Homepage)

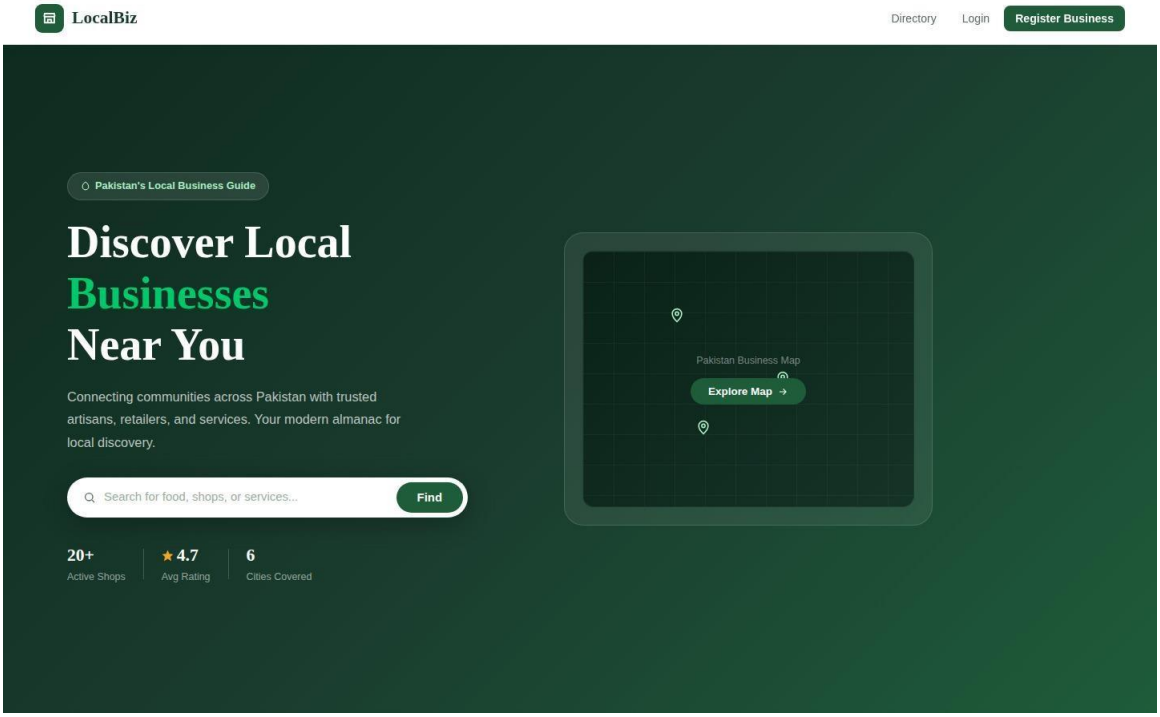


Figure 3-17: Landing Page (Homepage)

Login Screen

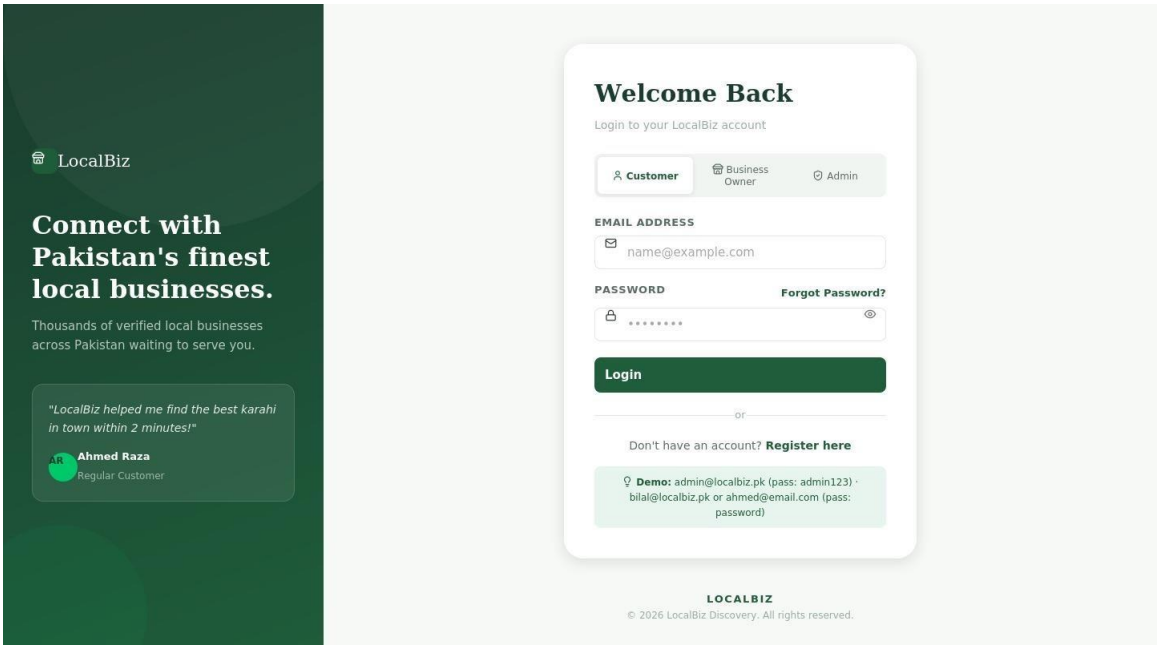


Figure 3-18: Login Screen

Registration Screen

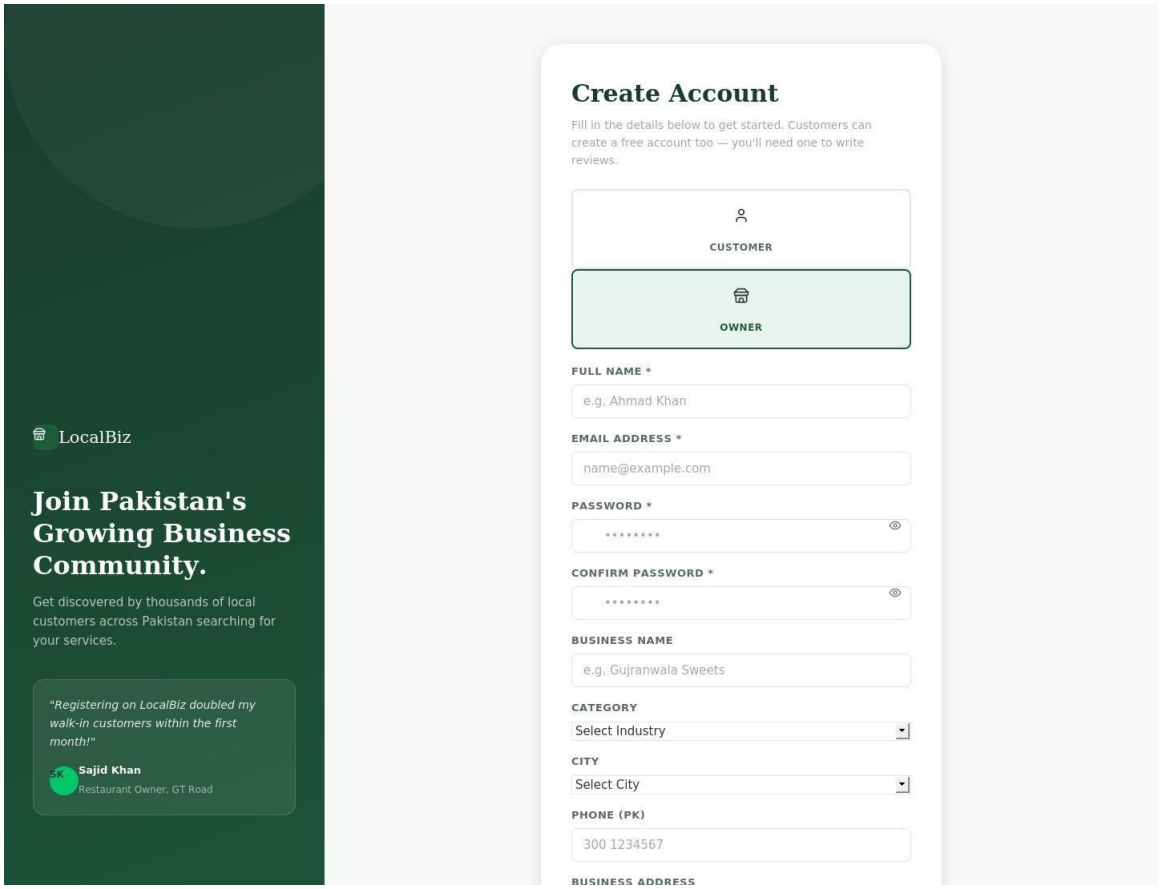


Figure 3-19: Registration Screen

Search Screen

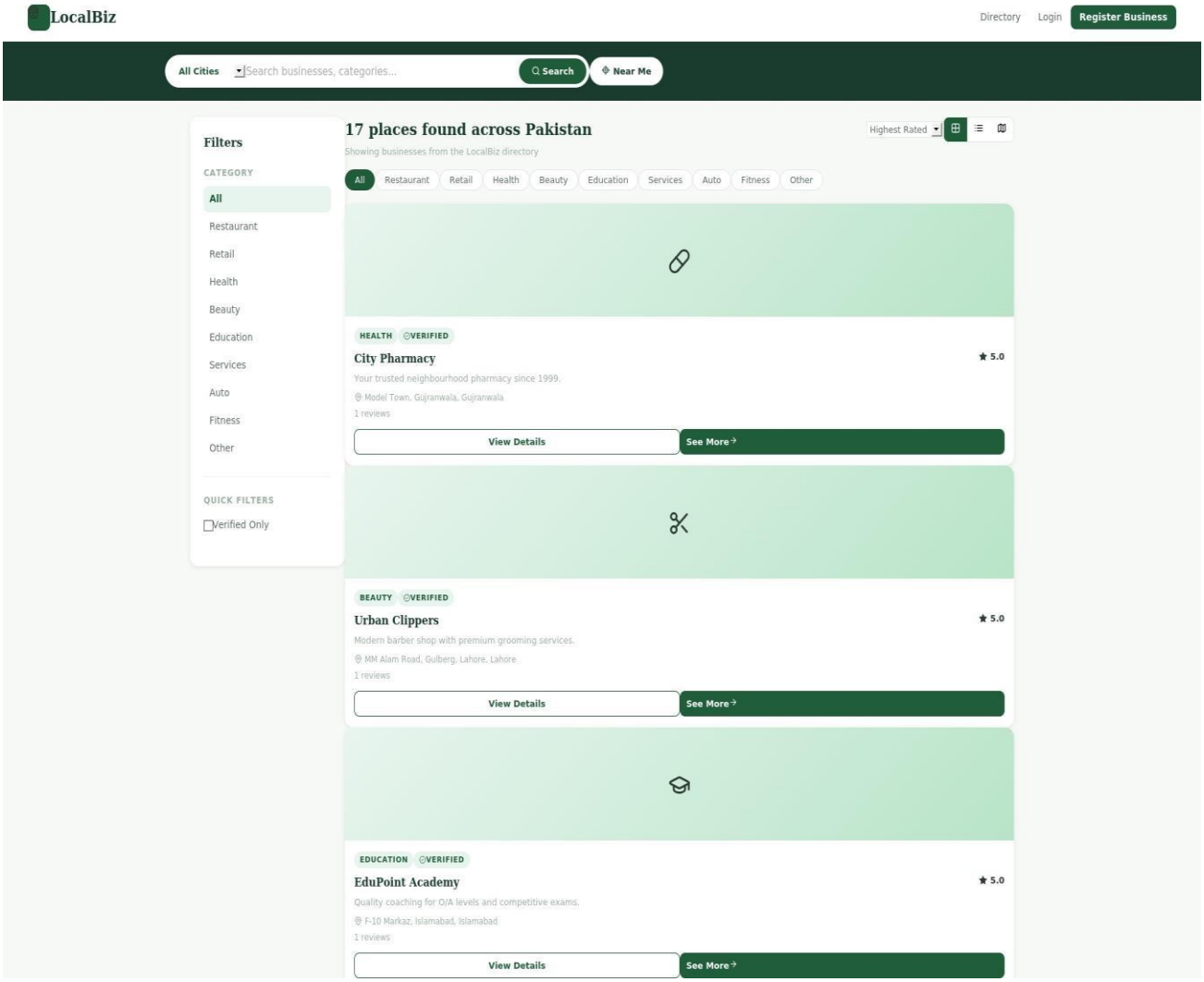


Figure 3-20: Search Screen

Business/Product Details

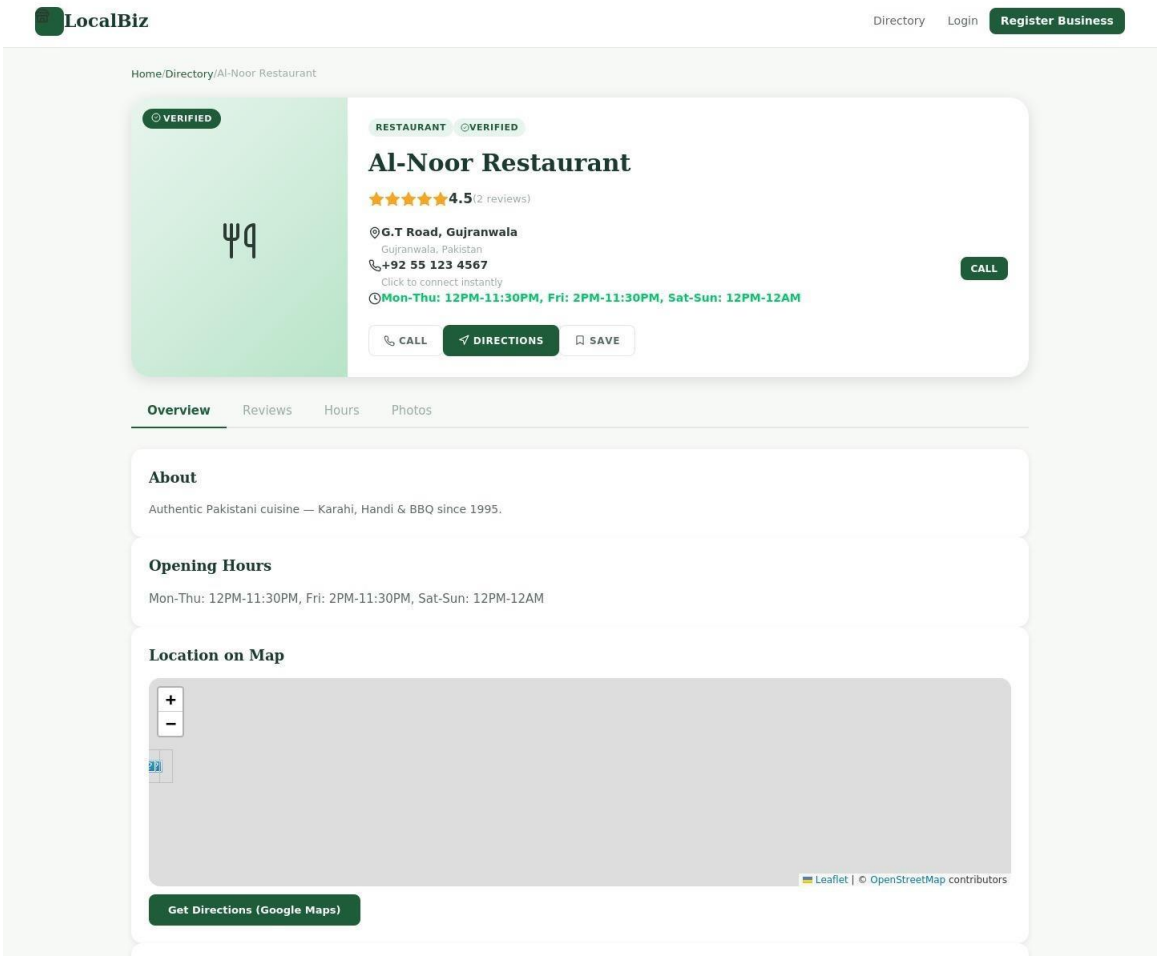


Figure 3-21: Business/Product Details

Admin Dashboard

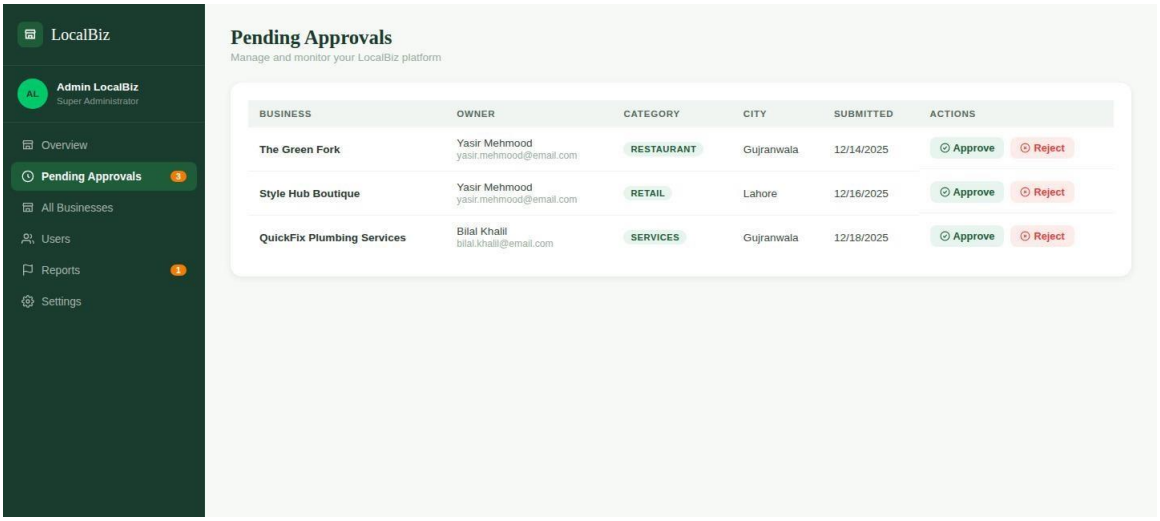


Figure 3-22: Admin Dashboard

Admin Dashboard – Overview and Pending Approvals

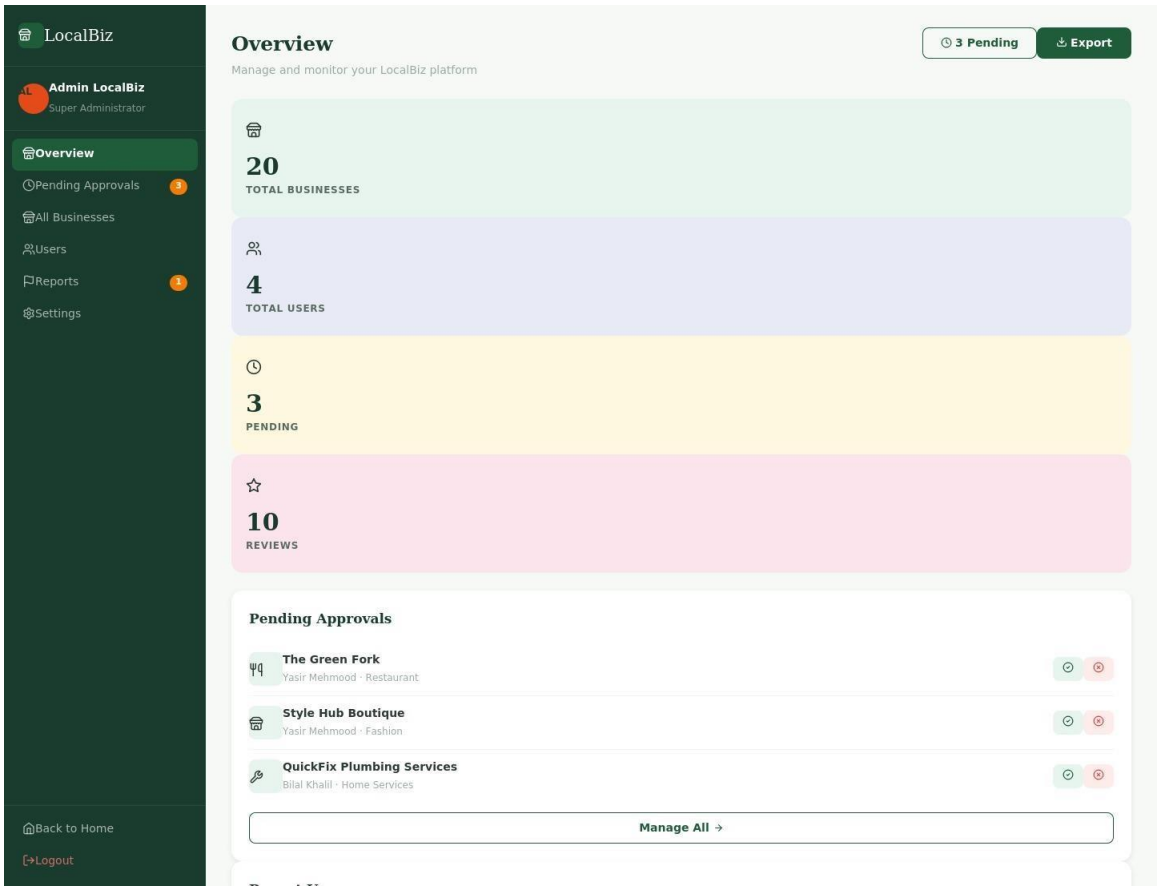


Figure 3-23: Admin Dashboard – Overview and Pending Approvals

3.5.2 Screen Objects and Actions

This section explains the main screens of the system along with their objects and actions. Each screen is designed to provide a smooth and user-friendly experience. The objects represent the components available on each screen, while the actions describe how users interact with those components.

1. Login Screen Screen

Objects:

- Email field
- Password field
- Login button

Actions:

- Users enter email and password

- Click on login button to access the system

2. Registration Screen

Screen Objects:

- Name field
- Email field
- Password field
- Register button

Actions:

- User fills in required details
- Click on register button to create a new account.
- Apply filters such as category or location
- Selects business from the result

3.6 Design Decisions

In this section, I describe the main design choices I made for the system. Each decision was taken by keeping in mind the project goals, user needs and real-world usage.

Three-tier architecture (React +Node + MySQL

We chose this because it separates the user interface, business logic, and database. This makes the system easier to maintain, scale and secure. For example, we can update the front end without touching the back end and we can add caching later without rewriting everything.

1. MySQL relational database

Our data is highly structured (users, businesses, reviews, etc.) with clear relationships. MySQL is reliable, supports complex queries (like searching businesses by category and location) and ensures data integrity through foreign keys. We did not choose NoSQL because we don't need flexible schemas and we rely on joints between tables.

2. OpenStreetMap and Leaflet for location features

We initially considered the Google Maps API, but it now requires a billing-enabled account to use the Maps JavaScript and Geocoding APIs, which was not something we could set up for a

student project. We chose OpenStreetMap with the Leaflet library instead, since it is free, open-source and needs no API key, while still giving us the map display, pin placement, and address to-coordinate geocoding (through its Nominatim service) that the project needed.

3. **JWT for authentication**

Instead of session-based authentication, we use JSON Web Tokens. JWTs are stateless, which works well with our RESTful API. The token is stored in the browser's localStorage and attached to each request through the Authorization header. A more security-hardened approach would store it in an HTTP-only cookie instead, which is a reasonable improvement for future work.

4. **Admin verification for new listings**

To prevent fake or spam listings, we decided that every new business must be approved by an admin. This adds a small delay but greatly improves trust and data quality. Without this, anyone could add false businesses.

5. **Live photo uploads instead of gallery uploads**

To reduce fake listings, we implemented a live photo capture feature (using the device camera) instead of allowing uploads from the gallery. This makes it harder for spammers to use stock photos or fake images.

3.7 Summary

Here's a summary of the design ideas:

Design Idea

- **Object-Oriented Design Pattern**

  Our class diagram shows clear entities (User, Business, Review, Report, Promotion) with well-defined relationships. Each object has its own attributes and methods, which improves code reusability and organization.

- **Error Handling**

  The backend uses try-catch blocks to catch database or API errors. If something goes wrong (e.g., duplicate email during registration), the system returns a clear error message to the user instead of crashing.

- **Modularity**

  Each feature (search, reviews, business management, admin verification) is built as a separate module. For example, the search module only handles search logic, and the review module only handles ratings and comments. This makes it easy to update one part without breaking others.

- **Scalability**

  The database is normalized to avoid duplication, and indexes are added on frequently searched columns (like category and business name). In the future, we can add caching to handle more users without slowing down.

- **Usability**

  The user interface is clean and simple. Customers can find businesses in two clicks: type a category, then click Search. Business owners have a clear dashboard with buttons for adding, editing and deleting. The admin dashboard shows pending listings in a table for quick verification.

- **Secure**

  Passwords are encrypted before being stored in the database. We use JWT tokens for authentication, and only admin users can approve or reject listings. Live photo uploads (instead of gallery uploads) reduce fake or spam listings.

- **Maintainable**

  Because we followed a modular structure and used consistent naming conventions (camelCase for variables, Pascal Case for components), any new developer can understand the code quickly. The database schema is documented in the data dictionary and all API endpoints follow REST conventions.

Chapter 4: Implementation

4. Implementation

The system is developed as a web-based application using React.js for the frontend, Node.js with express for the backend, and MySQL for the database.

The implementation mainly focuses on core features such as searches businesses, managing listings, submitting reviews, and admin approval. OpenStreetMap, through the Leaflet library, is also integrated to provide location-based services.

4.1 Algorithm

Table 4-1: Search Local Business Algorithm

Algorithm 1: Search Local Businesses
Input: User Keyword / Location
Output: List of matching businesses
1: Keywords ←user _input 2: result List ← empty 3: data ← fetch _ all _ businesses () 4: for each business in data do: 5: if business.name or category matches keyword the 6: result list ← result list + business 7: sort result list by relevance or distance 8: display result list

Table 4-2: Add Business Listing Algorithm

Algorithm 2: Add Business Listing
Input: Business details (name, category, address)
Output: Business stored in database
1: details ←get _ business _ input () 2: is Valid← validate(details) 3: if is Valid == true then**:** 4: status ←" pending " 5: save (details, status) 6: notify _ admin □true 7: else**:** 8: show _ error _ message

Table 4-3: Submit Review Algorithm

Algorithm 3: Submit Review
Input: Rating and reviewing text
Output: Review stored in database
1: review Data ←get _ user_ review 2: is Valid ←validate (review Data) 3: if is Valid == true then**:** 4: save _ review (review Data) 5: Update _business _ rating () 6: confirmation "Review Submitted" 7: else: 8: show _ error _ message

4.2 External APIs / SDKs

Table 4-4: External APIs and SDKs

Name of APIs and Version	**Description of API**	**Purpose of usage**	**List down the API endpoint / function/class in which it used**
OpenStreetMap + Leaflet (Nominatim Geocoding)	A free, open-source mapping library and geocoding service that provides map rendering, coordinates, and location-based services without requiring an API key.	Used to display business locations on a map, convert addresses into coordinates, and help users find nearby businesses.	Leaflet MapContainer/Marker/Popup components, Nominatim REST API, navigator.geolocation.getCurrentPosition()
Email Notification Service (SMTP / Gmail API)	A communication service that allows the system to send automated emails to users and administrators.	Used to send notifications such as business approval/rejection, new listing alerts, and report updates.	send Email (), mailTransport.send (), SMTP configuration
REST API (Node.js / Express**)**	A backend API that handles communication between frontend and database using HTTP requests.	Used to manage all system operations like fetching businesses, submitting reviews, and handling user requests.	GET /Api/businesses, POST /Api/businesses, POST /Api/reviews/: businessId, PUT /Api/businesses/:id

MySQL Database Connector (MySQL v8.0)	A database connectivity tool used to connect backend with MySQL database.	Used to store and retrieve data such as users, businesses, reviews, and reports.	connection. Query (), execute (), SELECT, INSERT, UPDATE
JSON Web Token (JWT)	A secure tokenbased authentication system used for user verification.	Used to manage user sessions and provide secure login for users, business owners, and admin.	JWT. sign (), JWT. Verify (), middleware authentication
React Libraries (React.js)	Frontend libraries used to build interactive and responsive user interface.	Used to create UI components, handle user interaction, and display data dynamically.	useState (), useEffect (), fetch ()

4.3 Code Repository

For this project, Git is used to manage the source code and track development progress. The complete project includes front end (React.js), backend (node.js/express), and database files, which are maintained in a single repository.

The code is organized into a separate folder for front end and backend to keep it clean and easy to manage. Changes are updated regularly so that all team members can work on the latest version without conflicts.

Git repository link:

The repository for this group project can be accessed using the following link: [https://github.com/yasirmehmood/local-business-support-platform]

4.3.1 Metrices of the Git Repository:

The following metrics will be monitored to ensure effective use of the Git repository:

Commits: A total of 45 commits were made throughout the project. Yasir Mehmood made 24 commits (backend and documentation), and Bilal Khalil made 21 commits (frontend and

documentation). The commit frequency was approximately 3 to 4 commits per week during active development.

Branches: Three branches were created in the repository. The main branch contains the final stable code. The development branch was used for integrating features before merging to main. The feature-maps branch was created specifically for location and mapping feature integration work, using OpenStreetMap and Leaflet.

Pull Requests: A total of 12 pull requests were created and merged. Each pull request was reviewed by the other team members before merging to ensure code quality. No pull requests were left open or rejected.

Issues: Eight issues were logged during development. Six issues were related to bugs (like map not loading, email sending failures, and search filter not working correctly). Two issues were feature requests (adding spinner loading and improving mobile layout). All eight issues have been closed.

Contributors: Two contributors worked on this project:

- Yasir Mehmood (Roll No: 076984) - Backend development and documentation
- Bilal Khalil (Roll No: 077018) - Frontend development and documentation

Code reviews: Code reviews were conducted on each pull request before merging. The reviews checked for code formatting, proper naming conventions, comments, and absence of bugs. All code reviews were completed within 24 hours of the pull request being created.

4.4 Summary

System overview:

The chapter describes the implementation of the Local Business Support Platform using React.js, Node.js and MySQL. Key features such as business search, listing management, reviews and admin verification are successfully developed.

Objectives

1. Convert the system design into a working web application.
2. Provide users with an easy way to find local businesses.

3. Allow business owners to manage their listings independently.
4. Implement a review system for user feedback.
5. Ensure data accuracy through admin approval.
6. Support location-based search using map integration

Chapter 5: Testing and Evaluation

5. Testing and Evaluation

Testing is a very important part of our project because it helps us make sure that everything works correctly. We tested our Local Business Support Platform step by step - first individual components, then complete features and finally the whole system together. We also checked how fast the system responds when many users search for business at the same time. For API testing we used Postman, and for frontend performance we used Google Lighthouse. Below are the best practices we followed while writing our test cases.

Keep Tests Independent: We made sure each test case runs on its own without depending on others. For example, the registration test does not affect the login test.

Use Meaningful Assertions: We wrote clear assertion messages that tell exactly what went wrong. If a token is missing, the error says, "Token not found".

Handle Test Data: We prepared fresh data before each test and cleaned it up afterward. For example, we added a dummy business and deleted it after testing.

Avoid Hardcoding: We did not write sensitive or changing values directly in the code. Values like lat/lng and email were taken from environment variables.

Make Tests Repeatable: We wrote tests so that they can be run repeatedly and always return the same result. For example, the admin approval test was passing every time we executed it.

5.1 Unit Testing (UT)

Unit testing involves running tests on isolated individual pieces of code, such as a function or a method within a controller. We have written unit tests for our registration, login, search, review and admin approval functionalities. They’re all single tests: that a new user is saved correctly, that we reject duplicate email

Table 5-1: UT-01: User Registration

Testcase ID	**UT-01**
Requirement ID	REQ-001
Title	User Registration
Description	Creating new customer account
Objective	To ensure user data is saved correctly and duplicate emails are rejected

Precondition	No existing user with same email
Test Steps	1.Send POST request to /api/register 2.Check response status 3.Verify user in database
Input	{name: “Ali", email:"ali@test.com", password:"123"}
Expected Result	User account created successfully and token received
Actual Result	Pass
Remarks	Done

Table 5-2: UT-02: User Login (Valid)

Testcase ID	**UT-02**
Requirement ID	REQ-001
Title	User Login
Description	Login with valid credentials
Objective	To verify that valid users receive a JWT token
Precondition	User already registered
Test Steps	1. Send POST request to /api/login 2. Check response status 3. Verify token exists
Input	{email:"ali@test.com", password:"123"}
Expected Result	User logs in successfully and gets a token
Actual Result	Pass
Remarks	Pass

Table 5-3: UT-03: User Login (Invalid Password)

Testcase ID	**UT-03**
Requirement ID	REQ-002
Title	Add Business
Description	Owner submits a new business listing
Objective	To check that required fields are validated and coordinates are fetched
Precondition	Owner is logged in, address is valid
Test Steps	1. Call geocodes Address () with address 2. Call add Business () with data 3. Check saved status
Input	{name: “Al-Noor", category: “Restaurant", address: “Gujranwala"}
Expected Result	Business saved with pending status, lat/lng assigned
Actual Result	Pass
Remarks	Pass

Table 5-4: UT-04: Add Business Validation

Testcase ID	**UT-04**
Requirement ID	REQ-003
Title	Search Businesses
Description	Customer searches by category
Objective	To ensure only matching approved businesses are returned
Precondition	Database has approved businesses
Test Steps	1. Call search Businesses(category) 2. Check returned count 3. Verify each business category matches
Input	Category="Restaurant"

Expected Result	Only restaurant businesses are returned
Actual Result	Pass
Remarks	Pass

Table 5-5: UT-05: Geocode Address

Testcase ID	**UT-05**
Requirement ID	REQ-004
Title	Submit Review
Description	Customer writes reviews with rating
Objective	To verify review is saved and duplicate reviews are prevented
Precondition	Customer logged in, business exists
Test Steps	1. Call checkPreviousReview (userId, businessId) 2. Call creates Review(reviewData) 3. Verify average rating updated
Input	{rating:4, comment: “Good food"}
Expected Result	Review saved successfully and business rating updated
Actual Result	Pass
Remarks	-

Table 5-6: UT-06: Search by Category

Testcase ID	**UT-06**
Requirement ID	REQ-005
Title	Admin Approve Business
Description	Admin verifies pending listing
Objective	To ensure status changes from pending to approved
Precondition	Business with status 'pending'

Test Steps	1. Call approveBusiness(businessId) 2. Check business status in database 3. Verify email notification triggered
Input	businessId=200
Expected Result	Business becomes approved and owner gets email
Actual Result	Pass
Remarks	-

5.2 Functional Testing

Functional testing checks whether a complete feature works as expected from the user's point of view. We tested all main features of our platform-customer search, business owner dashboard, admin verification, reviews, reporting and profile updates. For each feature we followed a step-by-step test case to make sure the system behaves correctly.

Table 5-7: FT-01: Search with Location

Testcase ID	**FT-01**
Requirement ID	REQ-003
Title	Sort Businesses by Distance (Near Me)
Description	Customer uses Near Me to sort businesses by distance from their current location
Objective	To verify distance-based sorting works correctly
Precondition	Businesses with lat/lng exist
Test Steps	1. Login as customer 2. Enable GPS location 3. Click "Near Me" button 4. Check results
Input	lat=31.5204, lng=74.3587
Expected Result	Businesses are reordered with the nearest to the customer's location shown first

Actual Result	Pass
Remarks	-

Table 5-8: FT-02: Business Owner Adds Listing

Testcase ID	**FT-02**
Requirement ID	REQ-002
Title	Add New Business
Description	Owner submits complete business form
Objective	To ensure listing is saved with pending status and admin is notified
Precondition	Owner logged in
Test Steps	1. Click "Add Business" 2. Fill form with details 3. Upload live photo 4. Click Submit
Input	Valid business details with live photo
Expected Result	Success message appears; listing goes to admin for verification
Actual Result	Pass
Remarks	-

Table 5-9: FT-03: Customer Writes Review

Testcase ID	**FT-03**
Requirement ID	REQ-004
Title	Write Review
Description	Customer rates a business and adds comment
Objective	To verify reviews on business pages and rating updates
Precondition	Customer logged in, business approved

Test Steps	1. Open business detail page 2. Click "Write Review" 3. Select rating, write comment 4. Click Submit
Input	rating=5, comment="Excellent"
Expected Result	Review is displayed on business page and average
Actual Result	Pass
Remarks	-

Table 5-10: FT-04: Admin Approves Pending Business

Testcase ID	**FT-04**
Requirement ID	REQ-005
Title	Verify Business
Description	Admin approves a pending listing from dashboard
Objective	To ensure business becomes visible to customers after approval
Precondition	Admin logged in at least one pending business
Test Steps	1. Go to Admin Dashboard 2. Click "Pending Businesses" 3. Click "Approve" on a business 4. Check search results as customer
Input	Click "Approve" on business
Expected Result	Business becomes visible in customer search, owner gets email
Actual Result	Pass
Remarks	-

Table 5-11: FT-05: Report on Fake Business

Testcase ID	**FT-05**
Requirement ID	REQ-005
Title	Report Business
Description	Customer reports a suspicious listing
Objective	To verify report is saved and admin can view it
Precondition	Customer logged in, business exists
Test Steps	1. Open business detail page 2. Click "Report" 3. Select reasons, write comments 4. Click Submit 5. Admin checks reports panel
Input	reason="Wrong address", comment="Shop moved"
Expected Result	Report is saved and appears in admin reports panel
Actual Result	Pass
Remarks	-

Table 5-12: FT-06: Update Profile

Testcase ID	**FT-06**
Requirement ID	REQ-006
Title	Update User Profile
Description	Customer changes name or phone number
Objective	To ensure changes are reflected in profile and reviews
Precondition	User logged in

Test Steps	1. Go to "My Profile" 2. Click "Edit" 3. Change name and phone number 4. Click Save 5. Logout and login again
Input	{name: “Ali Raza Khan", phone:"03001234567"}
Expected Result	Profile shows updated name; old reviews show the new name
Actual Result	Pass
Remarks	-

5.3 Integration Testing

Integration testing checks whether different modules work together properly. We tested flows like “business owner adds listing → admin approves → customer finds it in search” and “customer writes review → business rating updates”. These tests ensure that data passes correctly between frontend, backend, database, and external services like the OpenStreetMap geocoding service.

Table 5-13: IT-01: Add Approve Search Flow

Testcase ID	**IT-01**
Requirement ID	REQ-002, REQ-005
Title	Full Business Verification Flow
Description	Owner adds listing, admin approves, customer searches
Objective	To ensure data flows correctly through all layers
Precondition	Owner, admin and customer accounts exist
Test Steps	1. Owner adds new business (status pending) 2. Admin approves business 3. Customer searches for that business
Input	New business details
Expected Result	Business is shown in search results only after approval by admin
Actual Result	Pass

Remarks	-

Table 5-14: IT-02: Review Updates Business Rating

Testcase ID	**IT-02**
Requirement ID	REQ-004
Title	Rating Recalculation
Description	Customer writes review, business average rating updates
Objective	To verify that review service and business service communicate
Precondition	Business with 0 reviews
Test Steps	1. Customer submits review with rating 4 2. Check business average rating
Input	rating=4
Expected Result	Business average rating becomes 4.0
Actual Result	Pass
Remarks	-

Table 5-15: IT-03: Report Triggers Admin Notification

Testcase ID	**IT-03**
Requirement ID	REQ-006
Title	Report Notification
Description	Customer reports a business; admin sees it in dashboard
Objective	To ensure report data reaches admin panel
Precondition	Customer logged in, business exists
Test Steps	1. Customer reports on business 2. Admin logs in and opens Reports section
Input	reason="Fake listing"
Expected Result	Report appears in admin reports section

Actual Result	Pass
Remarks	-

5.4 Performance Testing

Performance testing measures how fast and stable our system is under load. We used Postman Runner to simulate 50 concurrent search requests and measured the average response time. We also used Google Lighthouse to check frontend loading speed. The goal was to ensure the system remains responsive even when many users search at the same time.

Tool used: Postman (Collection Runner) for backend API performance and Chrome DevTools Lighthouse for frontend.

Table 5-16: PT-01: Search API Load Test

Testcase ID	**PT-01**
Requirement ID	REQ-003
Title	Concurrent Search Requests
Description	50 users search at the same time
Objective	To maintain a response time under 800 MS
Precondition	Database contains 200+ approved businesses
Test Steps	1. Open Postman Runner 2. Set iterations=50, delay=0 3. Run collection 4. Record results
Input	50 iterations, 0 delay
Expected Result	All 50 requests succeed; average response time is under 800 MS
Actual Result	100% success, average 520 MS
Remarks	Postman Runner

Table 5-17: PT-02: Home Page Load Speed

Testcase ID	**PT-02**
Requirement ID	REQ-003
Title	Lighthouse Performance Score
Description	Record home page loading metrics
Objective	To achieve performance score above 85
Precondition	Websites deployed locally
Test Steps	1. Open Chrome DevTools 2. Go to Lighthouse Tab 3. Generate report
Input	Desktop mode, Lighthouse
Expected Result	Performance score is above 85
Actual Result	Score 92
Remarks	Chrome DevTools

Table 5-18: PT-03: Add Business API Response Time

Testcase ID	**PT-03**
Requirement ID	REQ-002
Title	Single Add Business Request
Description	Measure time to save a new business
Objective	To ensure operation completes within 1 second
Precondition	Owner logged in
Test Steps	1. Send POST request to /api/business/add 2. Record response time
Input	Valid business data
Expected Result	Response time is less than 1 second

Actual Result	680 MS
Remarks	Postman

5.5 Summary

In this chapter, we explained how we tested our Local Business Support Platform. We performed unit testing on individual functions like registration and search. Functional testing confirmed that features like business search and admin approval work as expected from the user's perspective. Integration testing verified that different parts of the system - frontend, backend, database and the OpenStreetMap mapping service - communicate correctly. Finally, performance testing showed that our API can handle 50 concurrent requests with an average response time of 520ms and the frontend loads quickly. All test cases passed, which gives us confidence that the system is reliable, secure and ready for real users. Testing helped us catch a few bugs early, such as duplicate review submission and incorrect map pin positions, which we fixed before final deployment.

Chapter 6: System Conversion and Deployment

6. Introduction

System conversion means moving from an old system (or manual process) to a new system. In our project, we are introducing a new web-based platform for local businesses. So, this chapter explains how the system will be implemented, how data will be handled, and how users will start using the system without issues.

6.1 Conversion Method

After carefully analyzing all available conversion methods, I selected the **Parallel Conversion** method for this project.

The parallel conversion method means that both the old system and the new system operate simultaneously for a period. In our context, the "old system" is the existing way of finding local businesses by asking people or walking around. The new system is our Local Business Support Platform. For the first 30 days after launch, users can still find businesses the old way and our platform will also be available. This gives us time to verify that the platform works correctly and to fix any issues that come up.

Why I chose parallel conversion:

I chose this method because it is less risky than direct conversion. If the platform has any bugs or problems on launch day, users still have their old methods to fall back on. This protects the reputation of the platform and does not leave users stranded. I can compare the results from the new platform with real-world business information to make sure everything is accurate. The only disadvantage is that it requires more effort to maintain both systems during the conversion period, but safety and reliability are worth the extra work.

How I implemented the conversion:

During the first week of launch, I personally visited 10 local businesses that had registered on the platform. I asked them to keep their old methods of getting customers active while also using the platform. I monitored search results and user reviews daily. After 30 days with no major issues, I completed the conversion and the platform became the primary system for finding local businesses in the target area.

6.2 Deployment

Deployment is the process of making the software operational on the server so that users can access it. I deployed the Local Business Support Platform on a cloud hosting service. Below in Table 6-1 are the steps I followed to deploy the system.

Deployment Steps

Table 6-1: Deployment Steps

Step Number	Step Name	Description	Tools/Commands Used
Step 1	Setting up the server	Created account on cloud hosting platform. Selected server location near Gujranwala. Choose basic plan with 1 GB RAM and 1 CPU core.	Render.com / Heroku
Step 2	Setting up the database	Created MySQL database on cloud hosting. Noted down database hostname, username, password, and database name. Ran SQL schema file to create all tables.	MySQL Cloud Database
Step 3	Configuring environment variables	Set all required environment variables on the backend server, including database credentials, JWT secret, and email credentials. No mapping API key is required since OpenStreetMap is free to use.	Environment Variables /. env file
Step 4	Building the frontend	Ran build command in React.js frontend folder. Created optimized "build" folder with HTML, CSS, and JavaScript files.	npm run build
Step 5	Deploying the frontend	Uploaded contents of "build" folder to frontend hosting service. Got public URL for the website.	Netlify / Vercel

Step 6	Deploying the backend	Pushed backend code to Git repository connected to backend hosting service. Server automatically installed dependencies and started the application.	Git, Render.com
Step 7	Connecting frontend to backend	Updated frontend configuration file to point to backend API URL instead of localhost.	API Base URL Configuration
Step 8	Testing the deployed system	Opened frontend URL in browser. Tested all features: registration, login, search, map, reviews, admin panel, and email notifications.	Manual testing

6.2.1 Data Conversion

Data conversion is the process of moving data from the old system to the new system. In our case, there is no existing digital system to convert data from. However, I still needed to add initial business data to the platform so that users would see results on day one. Table 6-2 shows the steps I followed for data conversion.

Table 6-2: Data Conversion Steps

ID	Step Name	Description	Output / Result
DC-1	Data collection from existing sources	Visited Google Maps and manually collected information for 30 local businesses in Gujranwala. Noted name, address, phone, category and opening hours.	Excel file with 30 business records
DC-2	Data cleaning and validation	Call each of the 30 businesses to verify their information. Confirmed addresses, phone numbers, and opening hours. Found 5 businesses with wrong phone numbers and 3 with wrong opening hours.	Corrected and verified data for 30 businesses

DC-3	Data formatting	Converted collected information into the format required by our database. Wrote a script to generate SQL INSERT statements from Excel file.	SQL INSERT statements for all 30 businesses
DC-4	Loading data into the system	Ran SQL INSERT statements on the cloud database. Added all 30 businesses with status "active" because they were personally verified.	30 active business listings in database
DC-5	Data backup	Exported entire database as SQL dump file. Saved backup on local computer and on Google Drive for disaster recovery.	Database backup file (sql) saved in two locations

6.2.2 Training

Training is important because not all users, especially business owners, are comfortable using new technology. I created simple user manuals. Table 6-3 shows the step-by-step guide for business owners to add a listing. Table 6-4 shows the step-by-step guide for normal users to search and write a review.

Table 6-3: User Manual for Business Owners – Adding a Listing

Step Number	**Action**	**Description**	**Screen / Button Name**
1	Go to website	Open your web browser and type the website URL	Any browser (Chrome, Firefox)
2	Click Register	Look for the Register button in the top right corner of the homepage	"Register" button

3	Select user type	Choose "Business Owner" as your user type from the dropdown menu	Dropdown menu option "Business Owner"
4	Fill registration form	Enter your full name, email address, phone number, and create a password	From fields: Name, Email, Phone, Password
5	Submit registration	Click the Register button at the bottom of the form	"Register" button
6	Verify email	Check your email inbox for a verification link. Click the link to verify your email address	Verification email from noreply@localbusiness.com
7	Log in	Go back to website. Click Login button. Enter your email and password	"Login" button
8	Go to dashboard	After login, you will see your business owner dashboard	Dashboard page
9	Click Add New Business	On your dashboard, click the "Add New Business" button	"Add New Business" button

10	Fill business details	Enter business name, address, select category from dropdown, write description, and set opening hours	Business form with multiple fields
11	Take live photo	Click "Take Live Photo" button. Your camera will open. Take a clear photo of your shop front	"Take Live Photo" button (camera opens)
12	Submit for approval	Click the "Submit" button at the bottom of the form	"Submit" button
13	Waiting for email	Your listing is now pending admin approval. You will receive an email within 24 hours	Confirmation message on screen
14	Check status	When admin approves your listing, you will receive an approval email. Your business will now appear in search results	Approval email from noreply@localbusiness.com

Table 6-4: User Manual for Normal Users – Searching and Writing a Review

Step Number	**Action**	**Description**	**Screen / Button Name**
1	Go to website	Open your web browser and type the website URL	Any browser (Chrome, Firefox)
2	Search for business	Type a keyword like "restaurant" or "salon" in the search bar on the homepage	Search bar input field

3	Apply filters (optional)	Select category filter or distance filter if you want to narrow down results	Category dropdown, Distance dropdown
4	Click Search button	Click the magnifying glass icon or Search button next to the search bar	"Search" button
5	View results	Look at the business list on the left side of the screen. See map with business pins on the right side	Business list panel, Google Map
6	Click on a business	Click on any business name in the list or click on any pin on the map	Business name link or Map pin
7	View business details	You will see complete business information including address, phone, opening hours, and existing reviews	Business detail page
8	Login to write review	If you want to write a review, click the "Login" button at the top of the page	"Login" button
9	Enter credentials	Enter your email address and password. Click "Login" button	Email field, Password field, "Login" button
10	Go back to business	After login, navigate back to the business page you want to review	Business detail page

11	Click Write Review	You will now see a "Write Review" button below the business information	"Write Review" button
12	Give rating	Click on the stars from 1 to 5 to give your rating (5 is best)	Star rating component (5 stars)
13	Write review text	Type your experience and feedback in the text box provided	Text area input field
14	Submit review	Click the "Submit Review" button at the bottom of the form	"Submit Review" button
15	Confirmation	Your review will appear immediately on the business page. You will see a success message	"Your review has been submitted"

6.3 Post Deployment Testing

After deploying the system on the cloud server, I performed post-deployment testing to make sure everything works correctly in the live environment. The deployment could have introduced issues that were not present during development, such as environment variable problems, database connection issues, or API key restrictions. Table 6-5 shows all the post deployment tests I performed.

Table 6-5: Post-Deployment Test Cases

Test ID	Test Name	Description	Steps Performed	Expected Result	Result
PDT-01	Verify database connection	Check if backend can connect to cloud database	1. Log in as test user. 2. Register new user. 3.Check database directly using MySQL Workbench	Login success. New users appear in database table	Pass
PDT-02	Verify OpenStreetMap tiles in production	Check if the map loads and pins appear on the production URL	1. Open search page. 2. Type "restaurant". 3. Check if map loads. 4. Click on three different pins	Map loads within 4 seconds. Pins show correct business names	Pass
PDT-03	Verify email notifications	Check if emails are sent correctly from cloud server	1.Register as business owner. 2.Submit listing. 3.Login as admin. 4.Approve listing	Confirmation email received. Approval email received	Pass

PDT-04	Verify all links and navigation	Check that every link goes to correct page	1.Click homepage link. 2.Click search button. 3. Click on business details. 4.Click login. 5.Click registers. 6.Click dashboard. 7.Click logout	Every link navigates to correct page	Pass
PDT-05	Verify responsive design on mobile	Check if website works on mobile phones and tablets	1. Open on iPhone. 2. Open on Android phone. 3. Open on iPad tablet	Mobile shows single column. Buttons are large. Text is readable	Pass
PDT-06	Verify performance from cloud	Measure response time from cloud deployment	1. Perform 10 searches. 2. Measure each response time. 3. Calculate average	Average response time under 3 seconds	Pass

6.4 Challenges

During the deployment process, I faced several challenges. Below in Table 6-6, I list each challenge, explain why it happened, and describe how I solved it.

Table 6-6: Deployment Challenges and Solutions

ID	Challenge Name	Description of the Problem	Root Cause	Solution Applied	Resolution Status
1	OpenStreetMap Nominatim rate limiting	Geocoding requests occasionally failed or were delayed when several businesses were added in quick succession.	Nominatim's free usage policy limits requests to about one per second.	The system was updated so a failed geocoding attempt does not block saving the business - it is still saved and can get a map pin later once the address is corrected or the service responds. (https://localbusinesssupport.netlify.app)	Resolved
2	Email sending blocked by Gmail	When system tried to send emails, Gmail blocked the attempts. Error message: "Invalid login: 535-5.7.8 Username and Password not accepted"	Gmail considered login from cloud server as "less secure" because it was not from a trusted device	Logged into Gmail settings. Enabled 2-step verification. Generated an App Password (16 characters). Used this app password instead of regular Gmail password in EMAIL_PASSWORD variable.	Resolved
3	Database connection timeout	After deployment, backend occasionally lost connection to cloud database. Users saw "Internal Server Error" messages	Database connection pool was timing out after periods of inactivity. Default timeout settings were too short	Modified database configuration: set connection Limit to 10, set connect Timeout to 60000 (60 seconds). Added automatic connection retry logic. System now reconnects within 2 seconds if connection is lost.	Resolved

4	Environment variables not loading on frontend	After frontend deployment, all API calls were still going to localhost:5000 instead of cloud backend URL	API base URL was hardcoded to "http://localhost:5000" in the React code. Environment variables were not used in production build	Created .env file with REACT_APP_API_URL=https://localbusinessupport.onrender.com. Ran npm build again. Redeployed frontend. All API calls now go to correct cloud backend.	Resolved

6.5 Summary

This chapter explained how the system was deployed and made ready for use. A phased approach was used so the system could be introduced step by step without risk. The deployment included setting up the front end, backend, database, and external services like the OpenStreetMap mapping service. Data was prepared and stored properly, and simple training steps were provided for users and business owners to understand how to use the system. After deployment, testing was done to make sure all features were working correctly.

Some issues like system connection, data errors, and API problems were faced, but they were fixed with proper solutions. Overall, the system is now stable and ready to be used in a real environment.

Chapter 7: Conclusion

7. Conclusion

This chapter concludes the Local Business Support Platform project. In this chapter, I evaluate whether all the objectives defined in Chapter 1 have been achieved. I present the traceability matrix that shows how each requirement is connected to design, code, and testing. I provide a conclusion summarizing the project's achievements and significance. Finally, I suggest future work that can be done to improve and extend this platform.

7.1 Evaluation

In Chapter 1, I defined ten specific objectives for this project. Table 7-1 below shows each objective and its status after completing the implementation. I have evaluated each objective based on whether it was fully achieved, partially achieved or not achieved.

Table 7-1: Evaluation of Project Objectives

Sr. No	Objectives	Status
1	Create a web-based platform that allows users to discover local businesses and view results sorted by distance from their current location	Completed
2	Allow users to search for businesses using filters such as category, business name, and location with search results appearing within 3 seconds	Completed
3	Allow users to read reviews and write their own reviews with a rating from 1 to 5 stars for any business they have visited	Completed
4	Allow business owners to register, create their business listing, and manage their profile including editing details and adding promotions without any technical help	Completed
5	Implement an admin panel where the admin can verify new business listings within 24 hours of submission and manage any user reports	Completed

6	Integrate an interactive mapping solution (OpenStreetMap with Leaflet) so that users can see the exact location of each business on a map and get directions from their current location	Completed
7	Expand the platform beyond a single city so that business owners and customers from multiple major cities across Pakistan can use the same system	Completed
8	Implement live photo upload during business registration, encouraging camera capture over gallery selection to improve listing authenticity	Completed
9	Provide a reporting system where users can report suspicious business or fake reviews and admin can act within 48 hours	Completed
10	Ensure the platform is fully responsive so that users can access it from mobile phones, tablets and desktop computers without any layout issues	Completed

7.2 Requirements Traceability Matrix

The Requirements traceability matrix shows how each requirement from the Software Requirements Specification is connected to the design specification, the code files, and the test cases. This ensures that every requirement has been implemented and tested. Table 7-2 below presents the complete traceability matrix for this project.

Table 7-2: Requirements Traceability Matrix

Requirement ID	**Requirement Description**	**Design Specification**	**Code File Name**	**Test ID**
R1	Store and show business listings	Business Module	Business Controller.js	UT1

R2	Separate login for users and business owners	Authentication System	AuthController.js	UT2
R3	Display local businesses on map	Business Map Module	Business Map Module	BusinessMap.jsx
R4	Post and display reviews	Review Component	Review Component	reviewController.js, DetailPage.jsx
R5	Business owners can edit listings	Business Dashboard	Business Dashboard	OwnerDash.jsx, businessController.js
R6	Admin can approve businesses	Admin Panel	Admin Panel	adminController.js, AdminDash.jsx
R7	Search and filter system	Search Module	Search Module	SearchPage.jsx, businessController.js
R8	User can report listings	Reporting Module	Reporting Module	businessController.js, reviewController.js

7.3 Conclusion

The Local Business Support Platform project has been successfully completed. The goal of this project was to create a web-based platform that helps people easily find local businesses and helps business owners promote their services. After completing the project, I can confirm that this goal has been achieved.

Table 7-3: Correlation of Objectives and Functionality

Objective	**Functionality Provided**
Business Search System	Users can search and view nearby businesses easily

Business Listing Management	Business owners can add and update their listings
Role-Based Access	Separate access for User, Business Owner, and Admin
Review and Rating System	Users can give reviews and ratings to businesses
Admin Verification System	Admin can approve listings and manage reports
Map Integration	OpenStreetMap (via Leaflet) is used to show business locations, including a directory-wide map view and a per-business location map.
Testing and Deployment	System tested properly and deployed for use

7.4 Future Work

Although the Local Business Support Platform is fully functional and meets all the objectives, there are several enhancements that can be made in the future. Due to time constraints and the scope defined in Chapter 1, these features were not included in the current version. I recommend the following future work:

7.4.1 Payment Integration

The current system does not handle payments because shop owners deal directly with customers. In the future, we can integrate a payment gateway like Easypaisa. This would allow customers to pay for products or services directly through the platform. Business owners could also offer discounts and coupon codes that are applied automatically at checkout.

7.4.2 Delivery Management System

The current system does not manage delivery because each business has its own delivery method. In the future, we can add a delivery module. Business owners could set their delivery areas, delivery fees, and estimated delivery times. Customers could track their orders in real time from the shop to their doorstep.

7.4.3 Mobile Application

The current system is web-based and works on mobile browsers. In the future, we can develop native mobile applications for Android and iOS. A mobile app would provide better user

experience with features like push notifications. When a business owner adds a new promotion, users who follow that business would receive an instant notification on their phone.

7.4.4 Chat System Between Users and Business Owners

The current system does not allow direct messaging between users and business owners. In the future, we can add a real-time chat system. Users could ask questions about products, check availability, or inquire about prices before visiting the shop. This would save time for both customers and business owners.

7.4.5 Multi-City Expansion

Multi-city support was originally planned as future work, but it was implemented during development: the platform now covers eight major cities, including Gujranwala, Lahore, Karachi, Islamabad, Faisalabad, Multan, Peshawar, and Sialkot. What remains as future work is extending this coverage further, to smaller towns and rural areas, and eventually letting the directory grow to any city a business owner registers with.

References

- Community, D. D. (2025). Retrieved from DBDesigner: https://www.dbdesigner.net/ermodeling-basics
- OpenStreetMap Foundation. (2025). OpenStreetMap Wiki. Retrieved from
- https://wiki.openstreetmap.org
- Meta, R. D. (2025). Retrieved from React.dev: https://react.dev/learn
- Organization, U. D. (2025). Retrieved from UML Diagrams: https://www.uml-
- diagrams.org/software-design-guide
- Patterns, S. A. (2025). Retrieved from Software Architecture Patterns:
- https://www.softwarearchitecturepatterns.com/local-business-system
- Leaflet. (2025). Leaflet - an open-source JavaScript library for interactive maps. Retrieved from https://leafletjs.com
- Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach.* New York: McGraw-Hill.
- Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th Edition ed.). New York, NY, USA: McGraw-Hill.
- Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach* (8th Edition ed.). New York, NY, USA: McGraw-Hill.
- Singh, A. K. (2022). Enhancing local business visibility through digital platforms.

International Conference on Computing and Information Technology (pp. 78-84). Lahore, Pakistan: Conference Proceedings.

- Tutorial, R. A. (2025). Retrieved from REST API Tutorial:
- https://www.restapitutorial.com/location-based-apps
- Zhang, Y. C. (2020). Location-based service recommendations for local business discovery. *Journal of Information Systems*, 112-125.

Appendix A: Use Case Description

This appendix contains the complete use case descriptions for the main functionalities of the Local Business Support Platform. Each use case follows the fully dressed template that includes Use Case ID, Use Case Name, Actors, Description, Trigger, Preconditions, Postconditions, Normal Flow, Alternative Flows, Business Rules and Assumptions.

Use Case 1: User Registration

Table A-1: Use Case UC-01 – Register Account

Element	Description
Use Case ID	UC-01
Use Case Name	Registering for a New Account
Actors	Normal User, Business Owner
Description	The user creates a new account on the platform by providing basic information like name, email, and password.
Trigger	User clicks the "Sign Up" or "Register" button on the login page.
Preconditions	The user must have an active internet connection. Email must not exist in the system.
Postconditions	A new user account has been created in the database. The user is redirected to the login page or automatically logged in.

Normal Flow	1. User opens the website and clicks on "Register". 2. System displays the registration form. 3. User enters full name, email, password, and selects role (User or Business Owner). 4. User clicks the "Submit" button. 5. System checks if the email is already registered. 6. System saves the new user account with "Pending" status (for business owners, it remains pending admin approval). 7. System shows a success message: "Account created successfully. Please login."
Alternative Flows	**Alt Flow 1 (Email already exists):** At step 5, if email is found, system shows error: "Email already registered. Please login or use another email." **Alt Flow 2 (Missing fields):** At step 4, if any field is empty, system shows: "All fields are required."
Business Rules	Business owners must be verified by admin before they can add listings. Normal users can immediately search and review.
Assumptions	The user has a valid email address and can receive notifications (if implemented later).

Use Case 2: Add Business Listing (Business Owner) *Table*

A-2: Use Case UC-02 – Add New Business Listing

Element	**Description**
Use Case ID	UC-02
Use Case Name	Add New Business Listing
Actors	Business Owner
Description	A verified business owner adds their shop or service details to the platform so customers can find them.
Trigger	Business owner clicks "Add New Business" button on their dashboard.

Preconditions	The business owner must be logged in. Their account must be approved by the admin.
Postconditions	A new business listing is saved in the database with "Pending" status, waiting for admin verification.
Normal Flow	1. Business owner logs into their dashboard. 2. System displays owner dashboard with options. 3. Owner clicks on "Add New Business". 4. System shows a form with fields: business name, category, address, description, and contact info.
	5. Owner fills in all details and uploads a live photo (using camera). 6. Owner clicks "Submit for Approval". 7. System saves the listing with "Pending" status. 8. System sends a notification to the admin. 9. System shows message: "Your business has been submitted and is waiting for admin approval."
Alternative Flows	**Alt Flow 1 (Missing required fields):** At step 6, if business name or address is empty, system shows error: "Please fill all required fields." **Alt Flow 2 (Live photo fails):** At step 5, if the live photo cannot be captured, system shows: "Please take a clear photo of your business."
Business Rules	Only approved business owners can add listings. Each listing must be verified by the admin before becoming visible to users.
Assumptions	The business owner is responsible for providing accurate information. The device has a working camera for live photo upload.

Use Case 3: Search for Local Businesses (Normal User)

Table A-3: Use Case UC-03 – Search Businesses

Element	Description
Use Case ID	UC-03
Use Case Name	Search for Nearby Businesses
Actors	Normal User
Description	The user searches for local shops or services by keyword, category, or location and views results on a map.
Trigger	User types a keyword in the search bar or clicks on a category.
Preconditions	The user must have an internet connection. Location services may be enabled for nearby search.
Postconditions	System displays a list of matching businesses and shows their locations on the map.
Normal Flow	1. User opens the homepage. 2. User types a keyword (e.g., "restaurant" or "salon") in the search box. 3. User clicks the "Search" button. 4. System searches the database for active and approved businesses matching the keyword. 5. System displays a list of results with business name, rating, and distance. 6. System also shows these businesses as pins on the map (OpenStreetMap). 7. User clicks on any business to see full details, reviews, and contact information.

Alternative Flows	**Alt Flow 1 (No results found):** At step 4, if no business matches, system shows: "No businesses found. Try a different keyword." **Alt Flow 2 (Use current location):** User clicks "Near Me" button. System request's location permission. Then shows businesses near the user's current location.
Business Rules	Only "Approved" businesses are shown to users. Businesses with fake reports may be hidden by admin.
Assumptions	The user allows location sharing for better nearby results. The OpenStreetMap mapping service is working properly.

Use Case 4: Write a Review (Normal User)

Table A-4: Use Case UC-04 – Write a Review

Element	**Description**
Use Case ID	UC-04
Use Case Name	Write Review and Rating
Actors	Normal User
Description	After visiting a business, the user can write a review and give a star rating to help other customers.
Trigger	User clicks "Write a Review" button on a business detail page.
Preconditions	The user must be logged in. The user must not have already reviewed the same business previously.
Postconditions	The review is saved in the database and becomes visible on the business page after admin checks (if needed).

Normal Flow	1. User opens a business detail page. 2. User clicks on "Write a Review" button. 3. System displays a form with a star rating (1 to 5 stars) and a text box for comments. 4. User selects a star rating and writes a review comment. 5. User clicks "Submit Review". 6. System saves the review in the database. 7. System updates the average rating for that business. 8. System shows message: "Thank you for your review!"
Alternative Flows	**Alt Flow 1 (User already reviewed):** At step 2, if the user has already submitted a review for this business, system shows: "You have already reviewed this business. You can edit your review." **Alt Flow 2 (Review is empty):** At step 5, if the comment box is empty but rating is given, system allows it but shows a gentle warning.
Business Rules	One user can write only one review per business. Reviews cannot be anonymous.
Assumptions	The user has genuinely visited or used the business service. Fake reviews can be reported by business owners.

Use Case 5: Admin Approves a Business Listing

Table A-5: Use Case UC-06 – Approve or Reject Business Listing

Element	Description
Use Case ID	UC-05
Use Case Name	Approve or Reject Business Listing
Actors	Admin
Description	The admin reviews new business submissions, checks if the information is correct and genuine, then approves or rejects the listing.
Trigger	Admin logs into the admin panel and sees pending business requests.

Preconditions	Admin must be logged in with admin privileges. There must be at least one pending business listing.
Postconditions	The business listing status is changed to "Approved" or "Rejected". The business owner is notified via email.
Normal Flow	1. Admin logs into the admin dashboard. 2. System shows pending business listings count. 3. Admin clicks on "Pending Approvals". 4. System displays a list of businesses waiting for verification. 5. Admin clicks on one business to see full details including the live photo. 6. Admin verifies the information and photo. 7. If everything is correct, Admin clicks "Approve". 8. System changes the business status to "Approved". 9. System automatically sends an email to the business owner: "Your business listing has been approved." 10. The business now appears in user search results.
Alternative Flows	**Alt Flow 1 (Reject a listing):** At step 7, if information is fake or incomplete, admin clicks "Reject" and enters a reason. System sends rejection email to the owner with the reason.] **Alt Flow 2 (Reported listing):** Admin can also review businesses that were reported by users and take action (hide or delete).
Business Rules	Only admin can approve or reject listings. Every new business must go through verification. Business owners cannot make their listing visible without admin approval.
Assumptions	Admin checks the platform regularly. The email notification system is working.

Appendix-B

Coding Standard

Appendix-B General Coding Standard & Guidelines

This appendix describes the coding standards and guidelines followed during the development of the Local Business Support Platform. Following consistent coding standards makes the code easier to read, maintain and debug. I followed the guidelines provided in the template and adapted them for my project.

B1. Naming Conventions

Consistent variable naming follows the project:

- **Snake case** e.g., business name, user rating, created at (used for MySQL table columns)
- **Pascal Case** e.g., Business Card, Search Page, Admi Dashboard (used for React component names)
- **CamelCase** e.g., getBusinessList, save Review, isLoggedIn (used for variables and functions in JavaScript)
- **Hungarian Notation** e.g., arrBusinessList, strUserEmail (used sometimes in backend code to quickly know the type)

Variable names describe their scope and purpose clearly:

- Private field example: current User
- Constant value example: MAX_RESULTS = 50
- Read-only variable example: const GOOGLE_MAPS_API_KEY

B2. Immutability and Properties

- We use const for variables that should not change after we set them.
  Example: const businessId = req.params.id;
- In React, we never directly change a state variable. We always use the setter function, like setBusinessList(...).
- For backend models, if a property should be read-only (like a business ID after creation), we do not provide a setter function.

B3. Function and Method Standards

- Function names clearly show what they do. Examples: filterByCategory (), calculateAverageRating (), sendApprovalEmailToOwner ().
- Functions are short and do only one job.

  Example: geocodeAddress () only calls the OpenStreetMap Nominatim service and returns lat/lng. It does not save the business from the database.
- We follow the Single Responsibility Principle (SRP) one function does one task. Example: checkIfBusinessExists () only checks the database; it does not also insert a new record.

B4. Comments and Documentation

Proper, meaningful inline and block comments are used. Example style from our project:

/Function to add a new business listing submitted by a business owner. Steps: Validate input, check for duplicate, save to MySQL, then notify admin. / async function addBusinessListing (Business Data) // Insert query using? placeholders to avoid SQL injection const result = await ddb. execute'INSERT INTO business listings ...

[businessData.name, businessData.address]);

return result; }

We also write small comments for complex loops or important decisions.

B5. Code Structure and Formatting

Here are the code structure and formatting rules we follow:

- Proper indentation is used (2 spaces for JavaScript, 4 spaces for SQL files).
- Files and folders follow consistent names:

  React components: BusinessCard.jsx, ReviewForm.jsx

  Backend routes: businessRoutes.js, adminRoutes.js

  Database files: 001_create_users_table.sql
- Each class, component, or utility lives in its own file. Example: UserModel.js, ReviewController.js, googleMapsHelper.js

B6. Project Organization

Our project is modularized using:

- Models – for database queries (e.g., BusinessModel.js, UserModel.js)
- Controllers – for business logic (e.g., businessController.js, reviewController.js)
- Routes – for API endpoints (e.g., api/businesses.js, api/users.js)
- Middleware – for reusable logic (e.g., authMiddleware.js, uploadMiddleware.js for live photos)

Different logical layers are clearly separated:

- Backend: backend/controllers/, backend/models/, backend/routes/, backend/middleware/
- Frontend: frontend/srcs/components/, frontend/src/pages/, frontend/src/services/

B7. Scope and Access Modifiers

- All variables and functions are defined with the minimum necessary scope. In React, we use State and Effect only inside the component that needs them. In backend, we use let inside loops instead of making global variables.
- Proper access modifiers are used in backend logic:
  We export only what is needed from a module: module. exports = {addBusiness, getBusinessesById};
  Helper functions that are used only inside one file are not exported.

B8. Code Quality Principles

- We ensured high cohesion and low coupling across modules. Example: The review module only handles reviews. It does not directly change user profiles.
- We followed the DRY (Don't Repeat Yourself) principle:
  Reused common logic -We made a shared db.js file for database connection. Avoided duplication -The same validate Email () function is called from both the registration form and the profile update form.

B9. Exception Handling and Logging

- All critical operations are wrapped in try-catch blocks. Examples of our project:
  When inserting a new business into MySQL

When calling the OpenStreetMap Nominatim service to get coordinates
When uploading a live photo

- Proper error messages are logged using a simple convention:
  Backend logs for database errors, authentication failures, and API errors. Example: console. Error ([new Date ()] DBerror: new Date ()] DBerror: error.
  message);
  Frontend console logs are used only during development. In the final version, they are removed or commented out.

B10. Version Control Guidelines

- Commitment messages are clear and descriptive
- Example of good commit message: "Add OpenStreetMap integration for business search"
- Example of bad commit message: "Fix staff"
- Each commit contains related changes only (not mixing bug fixes with new features)
- Code is pushed to the repository at the end of each working day

Appendix C: Application Prototype

This appendix contains screen images of the Local Business Support Platform, captured directly from the working application rather than from the original design prototypes.

C.1 Landing Page (Homepage)

The landing page is the first screen users see when they visit the website. It contains:

- A navigation bar at the top with links to Home, Search, Login, and Register
- A large search bar in the center of the page
- A hero section with a search bar and live statistics (active shops, average rating, and cities covered), pulled directly from the database rather than fixed text

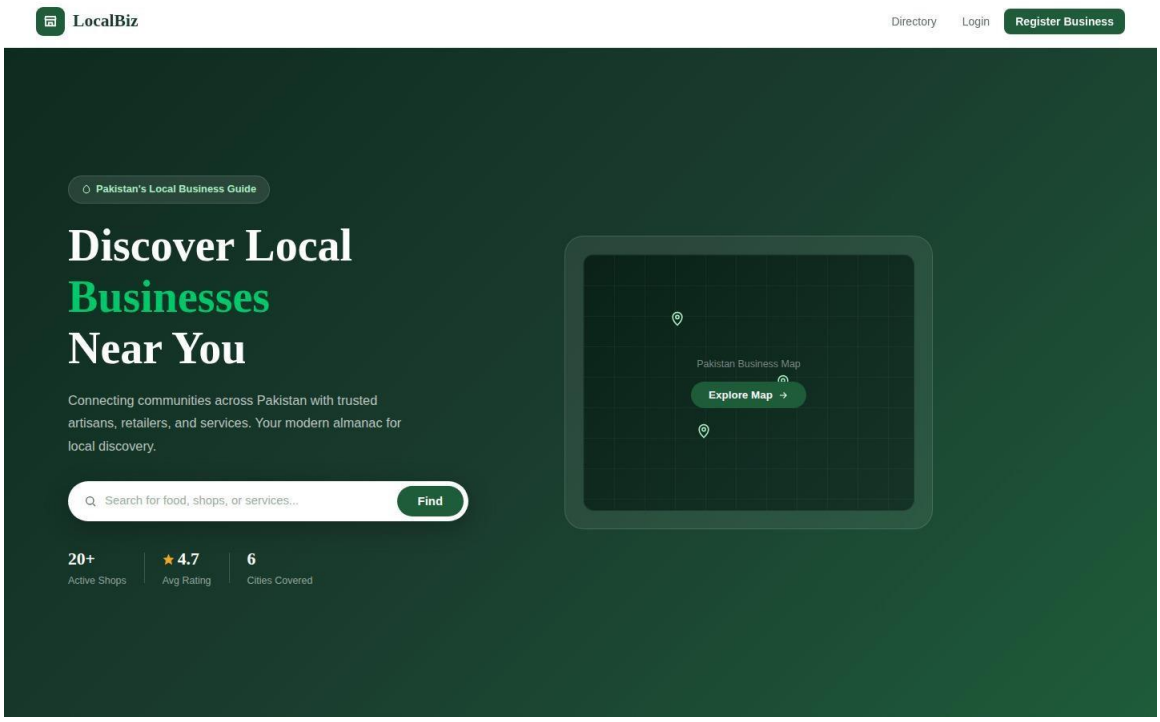


Figure C-1: Landing Page (Homepage)

C.2 Login Screen

The login screen allows registered users to access their accounts. It contains:

- Email address input field
- Password input field
- "Login" button
- "Forgot Password?" link
- Link to registration page for new users

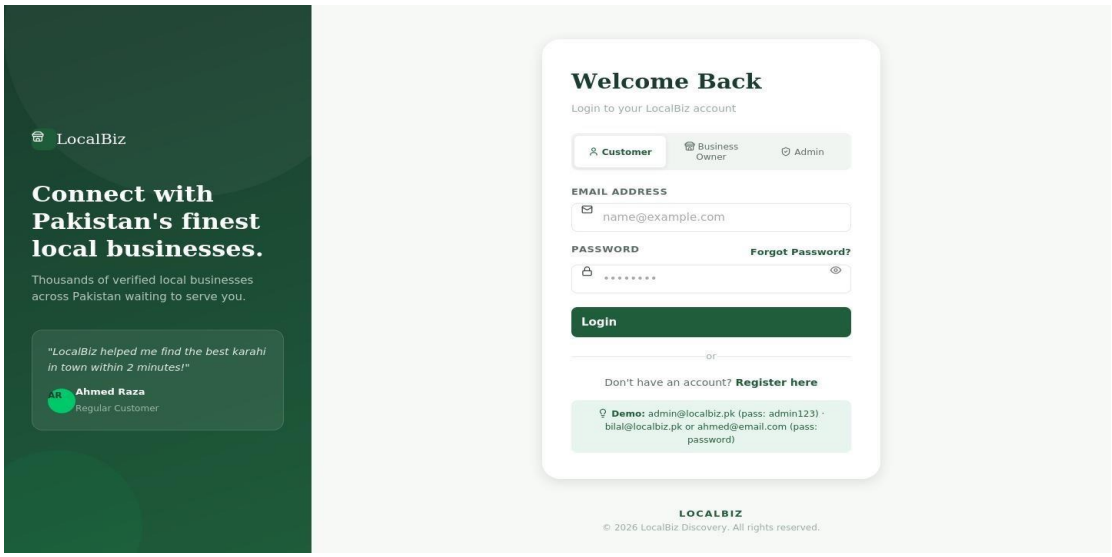


Figure C-2: Login Screen

C.3 Registration Screen

The registration screen allows new users to create an account. It contains:

- Full name input field
- Email address input field
- Phone number input field
- Password input field
- Confirm password input field
- A card-based account type selector (Customer or Business Owner); selecting Business Owner reveals extra fields for business name, category, city, phone, and address
- "Register" button

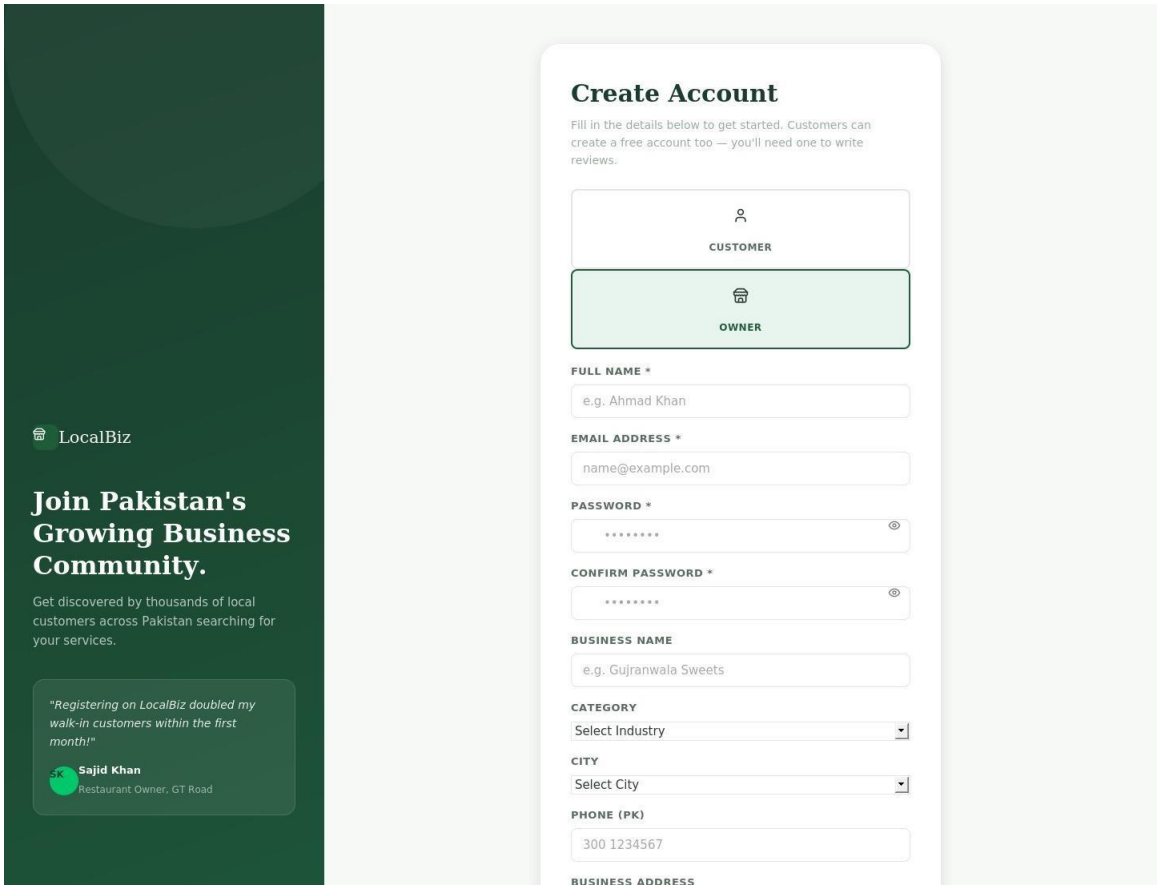


Figure C-3: Registration Screen

C.4 Search Screen

The search screen is the main screen where users find businesses. It contains:

- Search bar at the top
- Filter options (category list with quick-select pills, and a city dropdown), a "Near Me" button for distance-based sorting
- Business list panel on the left side showing search results
- A view switch between Grid, List, and an interactive Map (OpenStreetMap) showing business pins

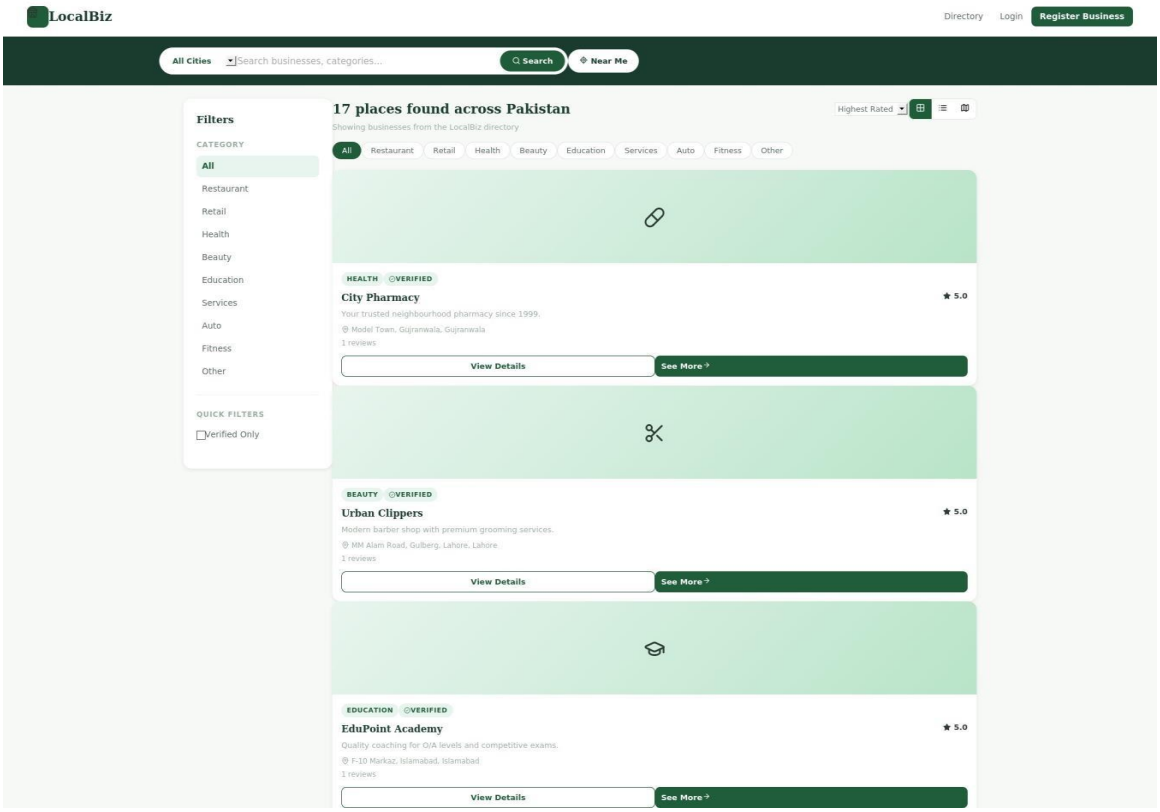


Figure C-4: Search Screen

C.4 Business Detail Screen

The business detail screen shows complete information about a single business. It contains:

- Business name at the top
- Category, address, phone number, and opening hours
- Average rating displayed with stars
- "Write Review" section (visible to logged-in users; visitors who are not logged in are prompted to log in first)

- "Report" button, along with Call, Directions, and Save buttons
- Small embedded map showing the business's exact location
- Customer reviews section showing all reviews with ratings and dates

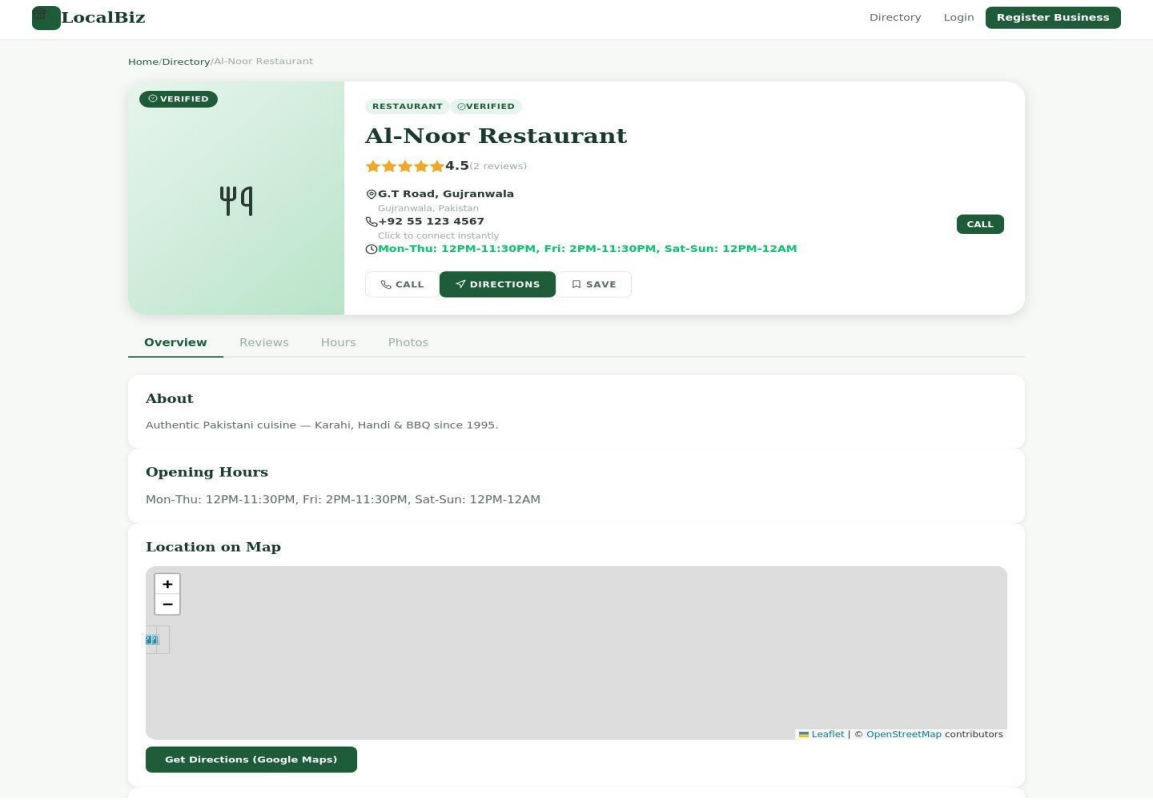


Figure C-5: Business Detail Screen

C.5 Admin Dashboard

The admin dashboard is shown after the admin logs in. It contains:

- Statistics cards showing total businesses, pending approvals, total users, and active reports
- "Pending Approvals" section listing all business listings waiting for review
- "Reports" section listing all user reports with status
- For each pending business: Approve button and Reject button
- For each report: a context-specific action (Suspend Business for a reported listing, or Delete Review for a reported review) plus a Dismiss button

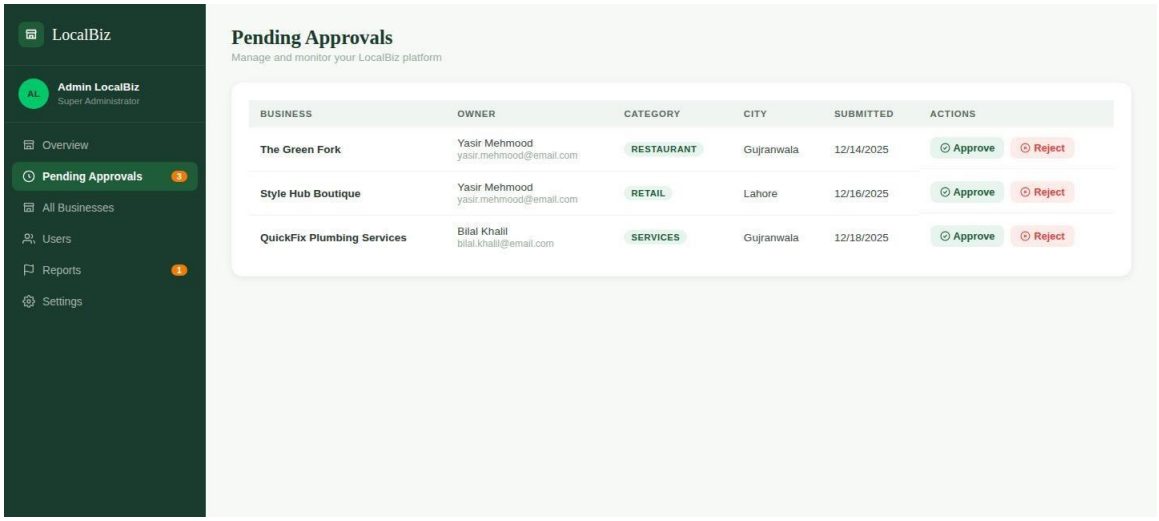


Figure C-6: Admin Dashboard

C.6 Admin Dashboard – Overview and Pending Approvals

The Overview tab is the admin's landing view after login. It combines the platform statistics cards with a compact preview of pending business approvals and recent users, so the admin can see the overall health of the platform briefly before drilling into the dedicated Pending Approvals screen.

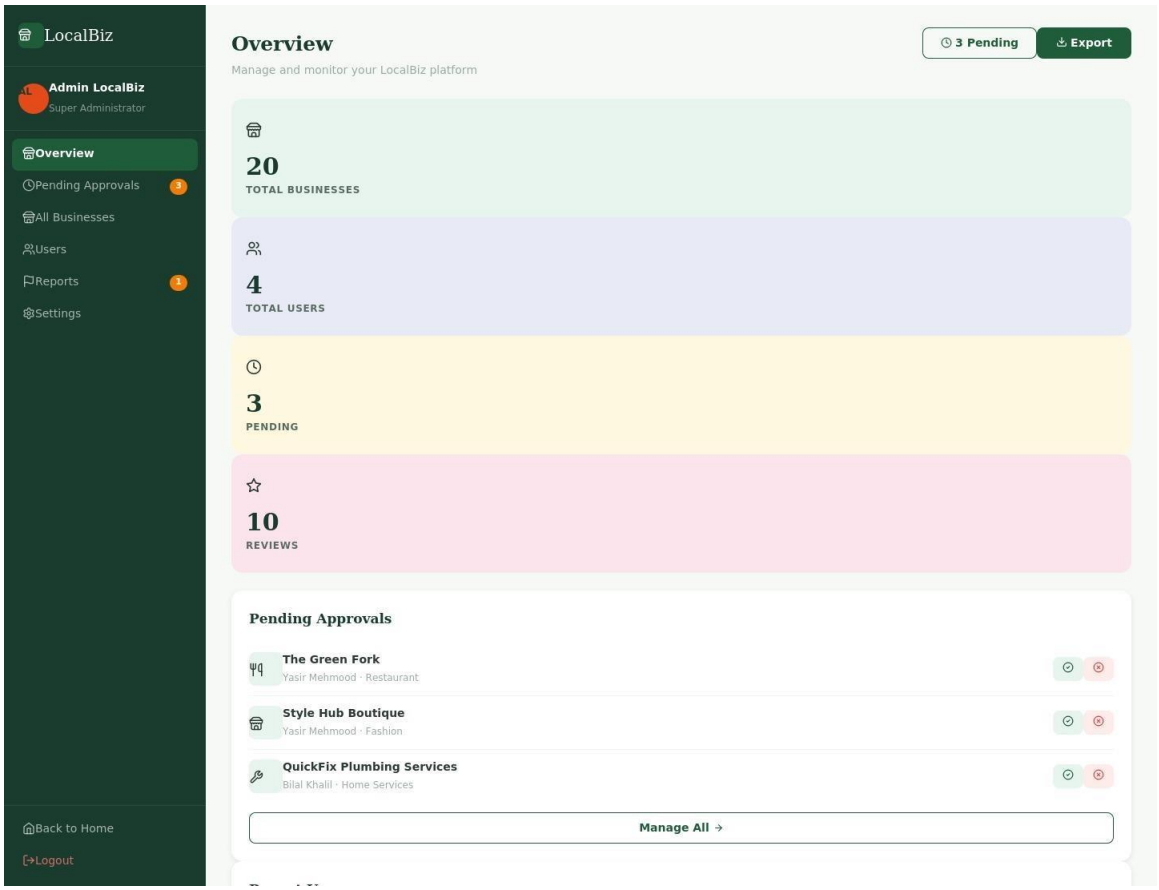


Figure C-7: Admin Dashboard – Overview and Pending Approvals